
STATISTICAL ANALYSIS IN THE MODERN ACADEMIA- STAMA

SPECIAL PROJECT

CHAIR OF APPLIED MATHEMATICS / FACULTY OF CIVIL ENGINEERING /
BAUHAUS-UNIVERSITY WEIMAR

YUVATEJA REDDY MARREDDY 127148

PETR VASENIN 126018

YASI FAIZI 127139

JOYSHREE MOZUMDER 127139

EXAMINERS

Dr. rer. nat. habil. Michael Schönlein

Prof. Sebastian Bock

Date of submission: July 18, 2025

Declaration

I hereby certify that the work embodied in the project is our own work, conducted under normal supervision. I confirm that the thesis contains no material which has been accepted, or is being examined, for the award of any other degree or diploma in any university or other tertiary institution. To the best of my knowledge and belief, the project contains no material previously published or written by another person, except where due reference has been made. We give consent to the final version of our thesis being made available worldwide when deposited in the University's digital repository, subject to applicable copyright law and any approved embargo.

Weimar, July 18, 2025

Yuvateja Reddy Marreddy 127148

Petr Vasenin 126018

Yasi Faizi 127139

Joyshree Mozumder 127139

July 18, 2025

Contents

1. Introduction	1
2. Data Source	5
Dataset Collection	5
1. Data from SCImago	5
2. Preparing and cleaning Dataset	5
3. Adding more Features	6
Metrics	6
Data Labels	8
Good Journals	8
Bad Journals	9
3. Exploratory Data Analysis (EDA)	10
1.Dataset Overview – What Are We Looking At?	10
2. Summary Statistics – How Do the Numbers Compare?	10
3. Correlation Analysis – What’s Related?	11
4. Outlier Detection	13
5. Clustering	14
6. Feature Importance – What Metrics Matter Most?	15
4. Classification Analysis	18
What We Did:	18
Models Trained:	18
5. Applying ML to the entire Dataset	20
6. Machine Learning Results and Citation Behavior Analysis in Specific Areas	23
Mathematics Journals	23
Domain-Specific Analysis	23
Model Performance Comparison	25
Economics	25
7. Conclusion and Future Work	26

Contents

A. Source code	27
Bibliography	77

1. Introduction

In recent years, the proliferation of academic journals has raised concerns about the integrity of scholarly publishing. While legitimate journals contribute to the advancement of knowledge, a subset—often labeled as predatory or problematic—exploits open-access models without proper peer review or editorial oversight. These journals distort citation ecosystems and undermine scientific trust.

- The goal of this project is to develop a data-driven framework to identify problematic journals using statistical and machine learning techniques. By analyzing a large dataset of over 27,000 journals, we aim to:
- Differentiate legitimate and problematic journals based on quantitative indicators.
- Identify key features most predictive of publication misconduct.
- Use unsupervised clustering to discover latent structures in the dataset.
- Apply supervised models to classify journals with high accuracy and explainability.

Our approach integrates traditional statistical hypothesis testing with modern machine learning pipelines. Through this fusion, we provide both interpretable and predictive insights, aiding academic institutions, libraries, and researchers in identifying credible publication venues.

Objective of the Study

The Objective of the Study is to develop a comprehensive framework for identifying and classifying problematic academic journals, particularly those that engage in predatory or questionable publishing practices. This study aims to:

1. **Identify Key Features:** Uncover critical characteristics, such as citation patterns, h-index, and SCImago Journal Rank (SJR), that differentiate legitimate journals from potentially problematic ones.
2. **Apply Statistical Testing:** Use hypothesis testing and statistical analysis to identify significant differences in metrics between legitimate and potentially problematic journals.
3. **Clustering Analysis:** Apply unsupervised clustering techniques to group journals based on their behavior, revealing latent patterns that

1. Introduction

could indicate suspicious or unethical publishing practices.

4. **Develop Classification Models:** Build and evaluate machine learning models to classify journals as legitimate or problematic, comparing the performance of models like Logistic Regression, SVM, and Decision Trees, with evaluation metrics such as accuracy, F1-score, and AUC.
5. **Provide Actionable Insights:** Offer insights to stakeholders (academic institutions, researchers, publishers) to help identify and classify journals with suspicious or unethical behavior, enabling better decision-making in academic publishing.

This approach integrates both statistical and machine learning methods to create a robust system for detecting and analyzing problematic journals in the academic publishing landscape.

Literature Review

There has been previous research on automated methods to determine journal quality. For example, Shen and Björk [2015] examined features of open-access journals and claimed features to differentiate good ones from bad ones. They analyzed citation habits, peer review, and publication charges and found that predatory journals had exceptionally low citation rates and fast turnaround times. Their method relied on manually selected features, which biased their results and limited their approach to scaling.

A related study conducted by Guillaume Cabanac and Magazinov [2021] proposed a machine-learning-based framework to identify predatory journals from textual analysis of paper metadata and reference sections. Their results showed promising accuracy for the model in removing fraudulent journals, but the performance was heavily dependent on the academic field.

Jingfeng Xia [2015] studied linguistic and editorial features of predatory journals, including quality of writing, article structure, and editorial board transparency. Their research showed that predatory journals often exhibit bad grammar, varying layouts, and dubious editorial boards. Although their approach works well for textual analysis, it misses deeper statistical regularities in citation distributions. Our method makes use of statistical properties instead of linguistic features, which makes the approach more general and less time-consuming.

Separately, Bohannon [2013] conducted an active experiment to submit scientific articles with faux-data to a range of open-access journals, in doing so

uncovering a more predatory publisher phenomenon where papers are accepted without proper peer review. His investigation revealed widespread ethical slip-ups in academic publishing but did not provide a systematic way to sample problematic journals at scale. As already mentioned, our approach does not require manual testing and is based only on statistical data.

Sample Metrics for Paper Quality in Academia

Bibliometric indicators are commonly used to measure the quality of academic journals and papers. Popular measures include:

- h-index : Hirsch [2005], a professor of physics at the University of California, introduced the h-index. It is a metric used to measure both the productivity and the citation impact of the publications of a scientist or a journal. Journals showing high h-index are widely regarded to publish impactful research.
- Impact Factor(IF) : The Impact Factor was developed by Baldwin [1964], a founder of the Institute for Scientific Information (ISI). It is calculated as the average number of citations received per paper published in a journal over a specific period (typically two years). High-impact journals are often considered more reputable, as they are frequently cited in the academic community. However, this metric has been criticized for being susceptible to manipulation, such as excessive self-citations and editorial policies that favor review articles.

While useful, these metrics can be gamed by predatory journals with self-citing and citation cartels. Thus, our approach acts as a complement to traditional journal assessment processes by allowing for data-driven clustering, used for a more definite detection mechanism in an automated fashion.

Structure of the Project

The Structure of the Project outlines the organization and flow of the work presented in this report. Below is a detailed description of the structure:

1. Introduction – This section introduces the problem domain, the importance of the study, and the scope of the project. It explains the motivation behind the research and outlines the goals and objectives of the study.
2. Dataset Description – In this section, the dataset used for the study is described in detail, including the number of journals analyzed, the

1. *Introduction*

features of the data, and any preprocessing or cleaning steps taken to prepare the data for analysis.

3. Exploratory Data Analysis (EDA) – This section presents the initial analysis of the dataset, including descriptive statistics and visualizations that reveal key trends, patterns, and outliers in the data. It helps in understanding the distribution of data and provides insights for further analysis.
4. Classification Analysis & Results – This section develops and evaluates supervised machine learning models to classify journals as legitimate or problematic. It compares the performance of different models, evaluates them using metrics like accuracy, precision, recall, F1-score, and AUC, and provides a thorough analysis of the results.
5. Conclusion and Future Work – The final section provides a summary of the project's outcomes, emphasizes key conclusions, and outlines possible directions for future research or improvements in the methodology.
6. References – A comprehensive list of all the sources cited throughout the report, including academic papers, datasets, and tools used during the study.
7. Appendices – Includes supplementary materials such as code snippets, extended result tables, and any other supporting information relevant to the project.

2. Data Source

In this chapter, we will report on the collection of all data from open sources that we used.

Dataset Collection

All sections of this part correspond to Jupyter files in repository GitHub.

1. Data from SCImago

SCImago Journal provides open databases for the years 1999-2024 with a large number of different metrics. We have selected metrics that we think are relevant in the context of "predatory and good" journals. The default SCImago dataset does not include metrics such as self-citations, number of cited and uncited documents, and estimated financial value, but these are publicly available when selecting a specific journal. Therefore, we decided to scrape these metrics for all journals in our dataset.

It should also be noted that, all data was collected for each year in the period 1999-2024. However, in the future, we will face the problem of missing data for 2024. When we reach this point, we will revisit this issue.

2. Preparing and cleaning Dataset

Here, we cleaned and prepared the data, specifically replacing absolute metrics with relative ones. For example, instead of the total number of Uncited Docs, we added the relative value of Uncited Docs per Total Docs. In this way, we wanted to make the metrics more correlated between big journals with a large number of published documents and small journals with a small number of documents.

We also cleaned the dataset of NaN values in some metrics and changed the data type in some metrics.

We encountered the problem of duplicate journals. This happened because the journals were published by different publishers and were therefore considered

2. Data Source

separate journals. We decided that the best solution would be to keep the first instance of the journal and delete subsequent duplicates (since the dataset is structured so that journals with the highest SJR are at the top, theoretically the least significant mentions will be deleted). This is not an ideal solution, as we are ignoring part of the journal's work, but since we are keeping the most significant instance, this should not have a major impact.

3. Adding more Features

Here we decided to add a couple more metrics, namely SNIP and IPP. We scraped them from Journal Indicators. Roughly speaking, these are the price for this journal, and this value correlates with the journal's impact.

However, we encountered a problem in that there was no data for these metrics for 1k journals. Therefore, we tried to fill in the missing data based on the available 26k using an ML algorithm. We tried different ones, but using Random Forest Regressor we get the best score. So it is about 89, which is a very good score, so we can safely fill in the missing data. Here we return to the problem from 1. Data from SCImago, namely the lack of data for 2024. SNIP and IPP are only available for 1999-2023. Therefore, in our study, we will use the most up-to-date data set that we have, i.e. for 2023.

Metrics

Sources of data: SCImago Journal, Journal Indicators

- **SJR** - The SJR is a size-independent prestige indicator that ranks journals by their 'average prestige per article'. It is based on the idea that 'all citations are not created equal'. SJR is a measure of scientific influence of journals that accounts for both the number of citations received by a journal and the importance or prestige of the journals where such citations come from. It measures the scientific influence of the average article in a journal, it expresses how central to the global scientific discussion an average article of the journal is.
- **H index** - The h-index expresses the journal's number of articles (h) that have received at least h citations. It quantifies both journal scientific output and scientific impact; it is also applicable to scientists, countries, etc. (See Hirsch, J.E. (2005)).
- **Total Documents** - Evolution of the number of published documents. All types of documents are considered, including citable and non citable documents.

- **Total Docs (3 years)** - Number of documents published by a journal in the three years prior to the selected year (documents from the selected year are excluded). For example, when year X is selected, documents published in years X-1, X-2, and X-3 are retrieved. All types of documents are considered, including both citable and non-citable documents.
- **Total Refs** - It includes all the bibliographical references in a journal in the selected year.
- **Self-cites/Total cites** - Evolution of the total number of citations and journal's self-citations received by a journal's published documents during the three previous years. Journal Self-citation is defined as the number of citation from a journal citing article to articles published by the same journal. (Number of citations received by a journal for documents published in the previous three years (i.e. citations received in year X to documents published in years X-1, X-2 and X-3). All types of documents are considered)
- **Citable documents** - Number of citable documents published by a journal in the three years prior to the selected year (documents from the selected year are excluded). Only articles, reviews and conference papers are considered.
- **Cites per Document (2 years)** - Average citations per document over a two-year period. It is calculated by considering the number of citations received by a journal in the current year for documents published in the previous two years. For example, citations received in year X for documents published in years X-1 and X-2.
- **Cited documents vs Uncited** - Ratio of a journal's items, grouped in three years windows, that have been cited at least once vs. those not cited during the following year. (only in the first year of the publication) (For example, the cited documents in year X represent the total cited documents in years X-1, X-2 and X-3.)
- **Estimated financial value** - It represents the potential financial worth of a journal. It is obtained by multiplying the journal's Estimated APC by the total number of citable documents published over the past five years. This value reflects the hypothetical revenue a journal could generate based on its estimated publication costs and scholarly output.
 - **Estimated APC** - It estimates the article processing charges (APCs) a journal might charge, based on its visibility, prestige, and impact as measured by the SJR. It does not reflect the actual APC, but rather a calculated approximation based on journal quality.
- **IPP** - the impact per publication, calculated as the number of citations given in the present year to publications in the past three years divided

2. Data Source

by the total number of publications in the past three years. IPP is fairly similar to the well-known journal impact factor. Like the journal impact factor, IPP does not correct for differences in citation practices between scientific fields. IPP was previously known as RIP (raw impact per publication).

- **SNIP** - the source normalized impact per publication, calculated as the number of citations given in the present year to publications in the past three years divided by the total number of publications in the past three years. The difference with IPP is that in the case of SNIP citations are normalized in order to correct for differences in citation practices between scientific fields. Essentially, the longer the reference list of a citing publication, the lower the value of a citation originating from that publication. A detailed explanation is offered in our scientific paper.

These features provide important information about the operational aspects of the journal, which is used to predict whether the journal meets academic standards or engages in predatory behavior.

Data Labels

Here we will talk about how we know which journal is "good" and which is "bad". In accordance with the sources listed below, we took the data, and the code of labeling itself is available in GitHub.

Good Journals

1. We initially received a list of **reputable publishers** from professors and marked all journals published by these publishers as "good". Here is a list of them:
 - 10.1007 Springer
 - 10.1186 Springer Nature (formerly BioMed Central for select journals)
 - 10.1016 Elsevier
 - 10.1080 Taylor and Francis
 - 10.1090 American Mathematical Society (AMS)
 - 10.1137 Society for Industrial and Applied Mathematics (SIAM)
 - 10.1515 De Gruyter
 - 10.3934 American Institute of Mathematical Sciences (AIMS)
 - 10.4171 European Mathematical Society (EMS) Publishing House
2. **Google Scholar's data.** We took all the top journals that were listed.

Bad Journals

1. We initially received a list of **predatory publishers** from professors and marked all journals published by these publishers as "bad". Here is a list of them:
 - 10.3390 Multidisciplinary Digital Publishing Institute (MDPI)
 - 10.14569 Science and Information Organization (SAI)
 - 10.18576 Natural Sciences Publishing (NSP)
 2. **Beall's list** All journals on this list are marked as "bad".
 3. **DOAJ database** This is the largest trusted list, so if a journal is not included here, it is potentially "bad".
- In total, we managed to label 8.7k journals.

3. Exploratory Data Analysis (EDA)

1. Dataset Overview – What Are We Looking At?

We start by understanding the types of journals in our dataset. They come from the sources we discussed in the “Data Labels” section.

Our main goal is to identify unusual or unethical behavior patterns among problematic journals, especially those trying to boost their metrics unfairly.

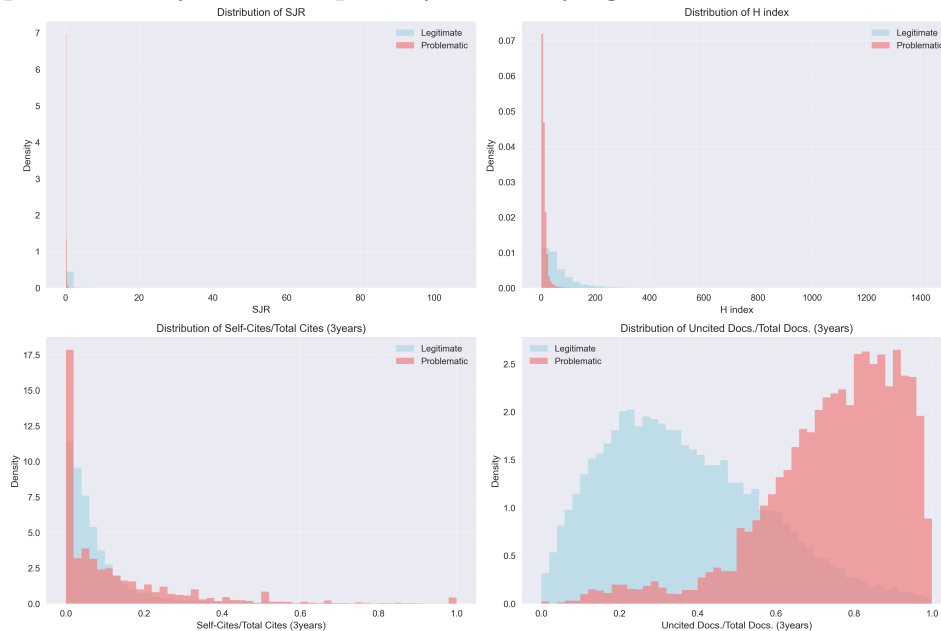


Figure 3.1.: Here, we compare distributions of metrics like citations and h-index between the two groups.

2. Summary Statistics – How Do the Numbers Compare?

Next, we look at summary statistics like the average and range of key metrics:

1. SJR (Scientific Journal Rank)
2. H-index

3. SNIP (Source Normalized Impact per Paper)
4. IPP (Impact per Publication)
5. Self-citation rates

Legitimate journals tend to have higher h-index and citation scores. Problematic journals often show odd spikes in self-citation or low impact metrics.

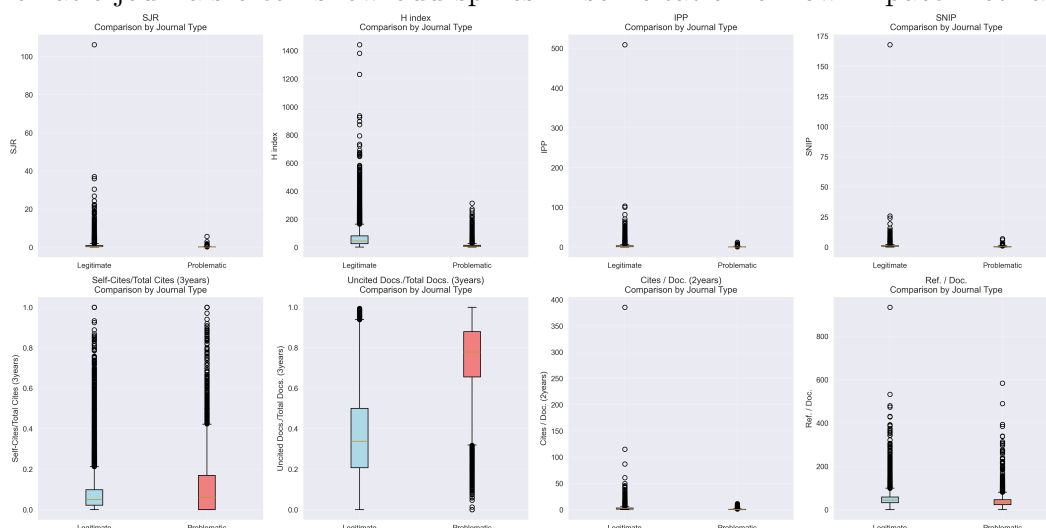


Figure 3.2.: Boxplots show how values like citations, h-index, and self-citations are spread across legitimate vs. problematic journals.

3. Correlation Analysis – What’s Related?

We want to see if certain metrics are linked. For example:

1. Do more citations always mean higher h-index? Not always, but mostly yes. There is a positive link between the number of citations and the h-index. So, if a journal gets more citations, it usually has a higher h-index — but it’s not guaranteed because the h-index also depends on how those citations are spread across articles.
2. Are journals inflating impact via self-citations? Doesn’t look like it. Self-citation ratio doesn’t have a strong connection with most impact metrics. That means just citing themselves a lot doesn’t really boost their overall score or reputation.

3. Exploratory Data Analysis (EDA)

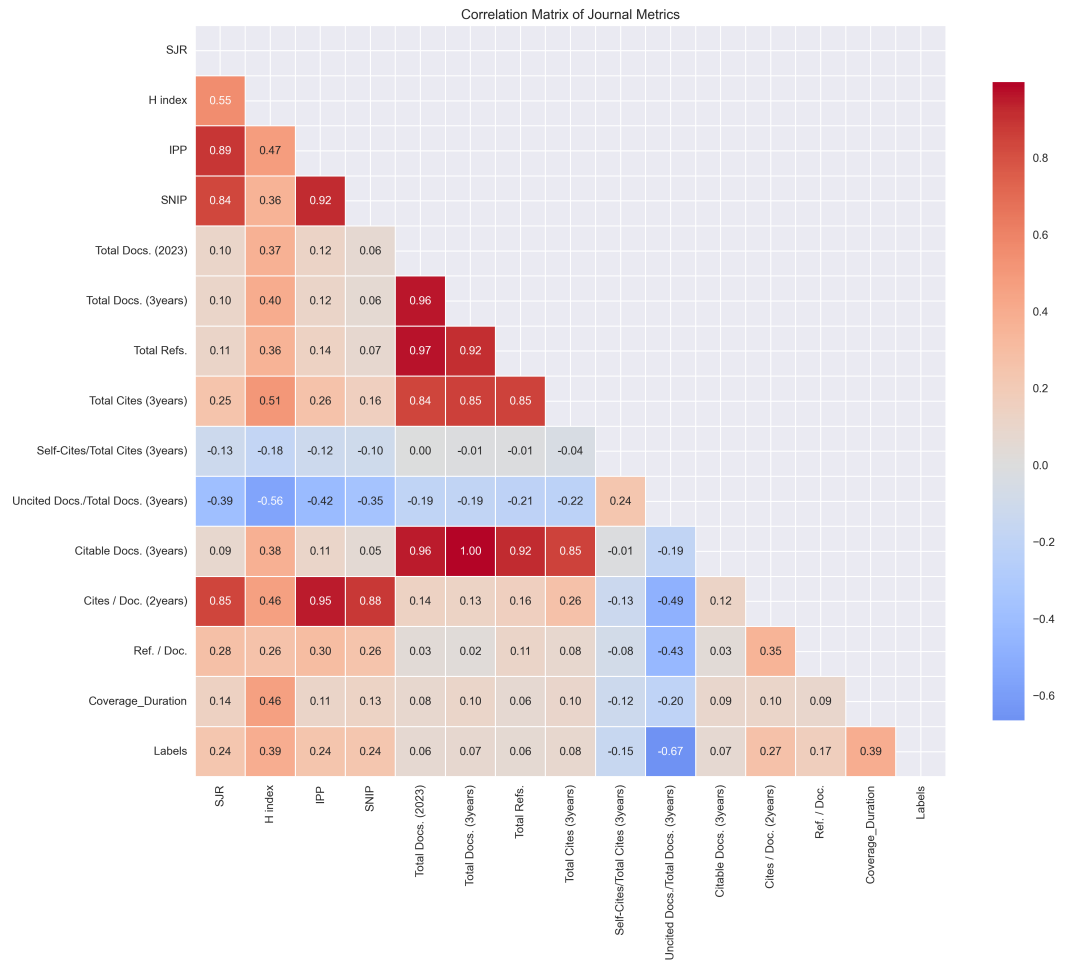


Figure 3.3.: This heatmap helps visualize how strongly metrics are correlated with each other.

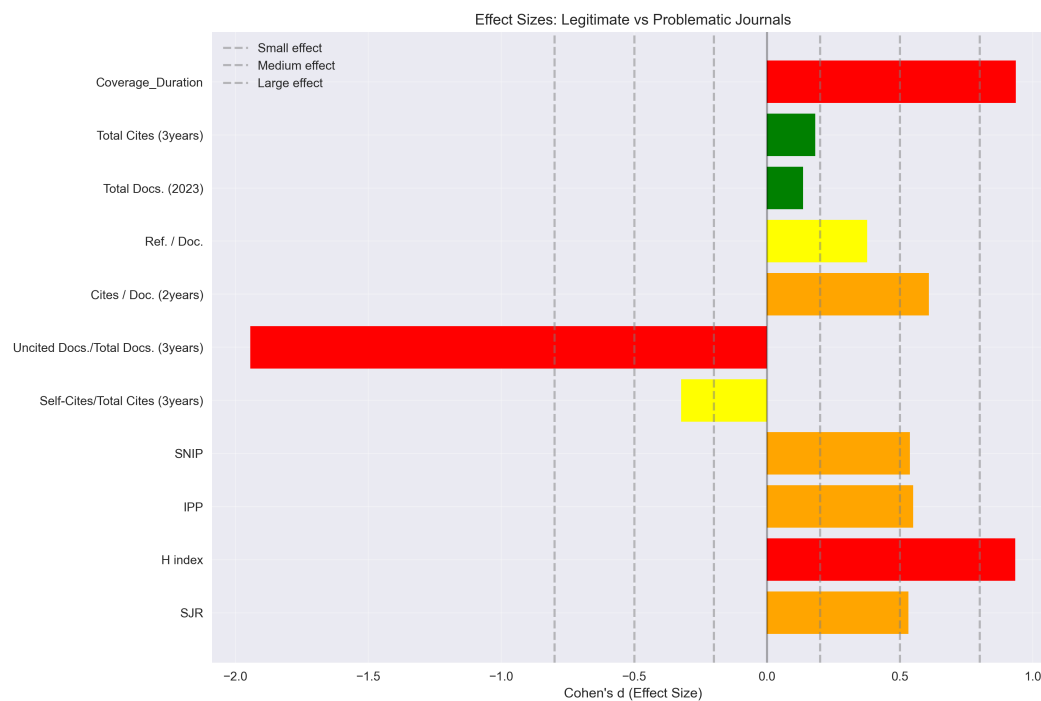


Figure 3.4.: This plot shows which features have the biggest difference between journal types, highlighting what really matters.

4. Outlier Detection

Some journals behave in ways that are far from the norm — for example:

1. Very high self-citation ratios
2. Almost no outside citations
3. High publication count but low engagement

3. Exploratory Data Analysis (EDA)

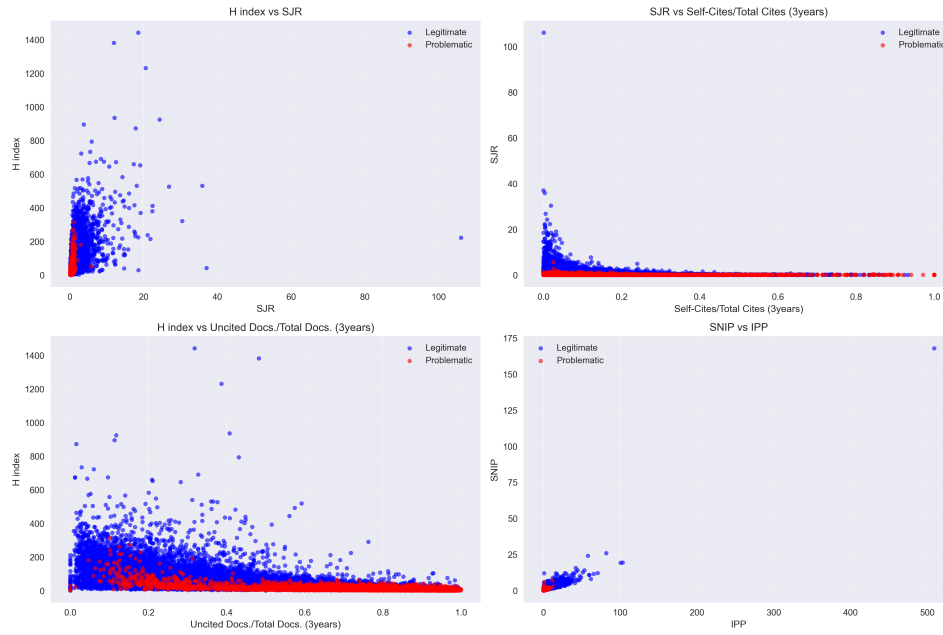


Figure 3.5.: This scatter plot helps spot suspicious journals that could be manipulating metrics.

5. Clustering

To simplify analysis, we use clustering to group journals by behavior. Journals with similar patterns (e.g., citation inflation, low h-index) naturally cluster together. This helps us spot patterns among problematic publishers.

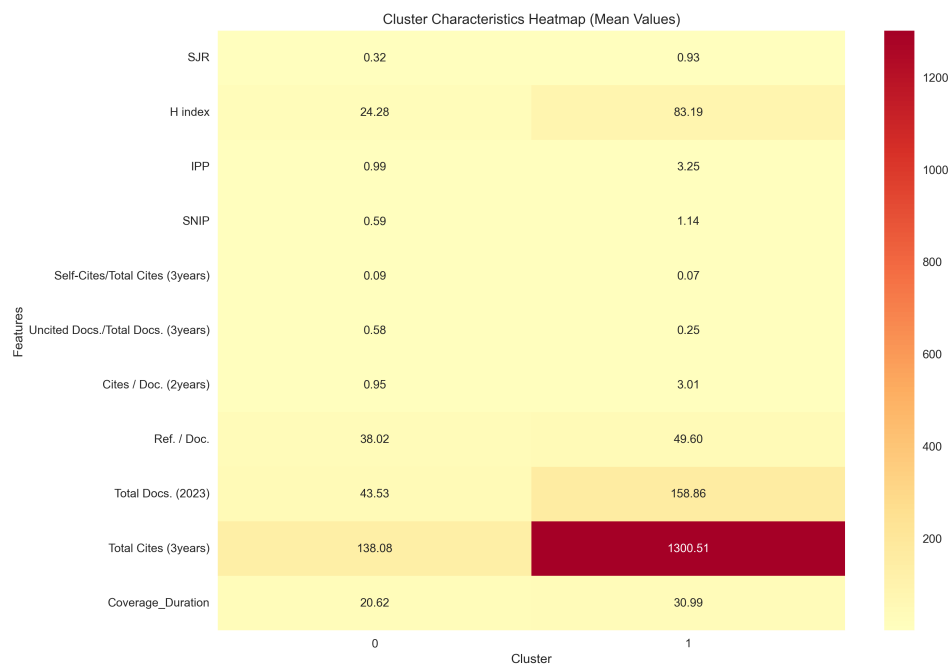


Figure 3.6.: Shows what makes each cluster unique – like high self-cites or low impact.

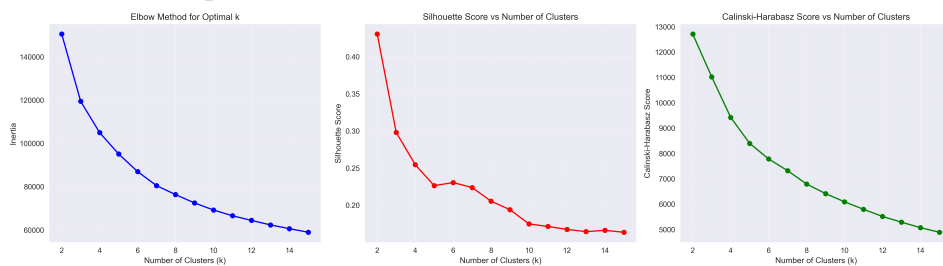


Figure 3.7.: Justifies how many clusters we chose using elbow or silhouette methods

6. Feature Importance – What Metrics Matter Most?

Before we move into classification or detection, it's useful to see which features are the strongest indicators of misconduct.

3. Exploratory Data Analysis (EDA)

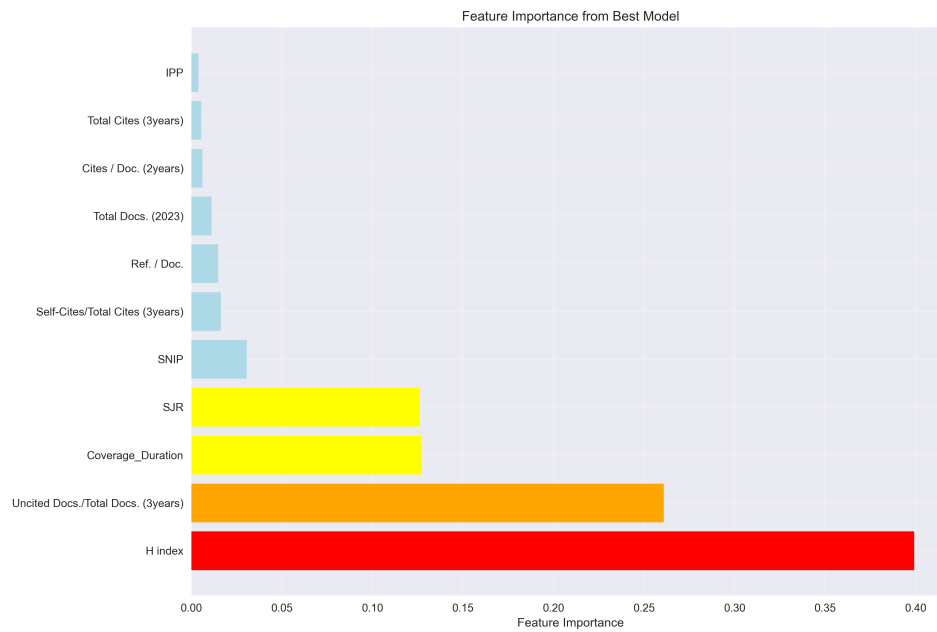


Figure 3.8.: This plot shows which individual features (like self-citation rate or the share of uncited documents) differ the most between legitimate and problematic journals. It helps us understand which metrics are most useful on their own to flag suspicious journals.

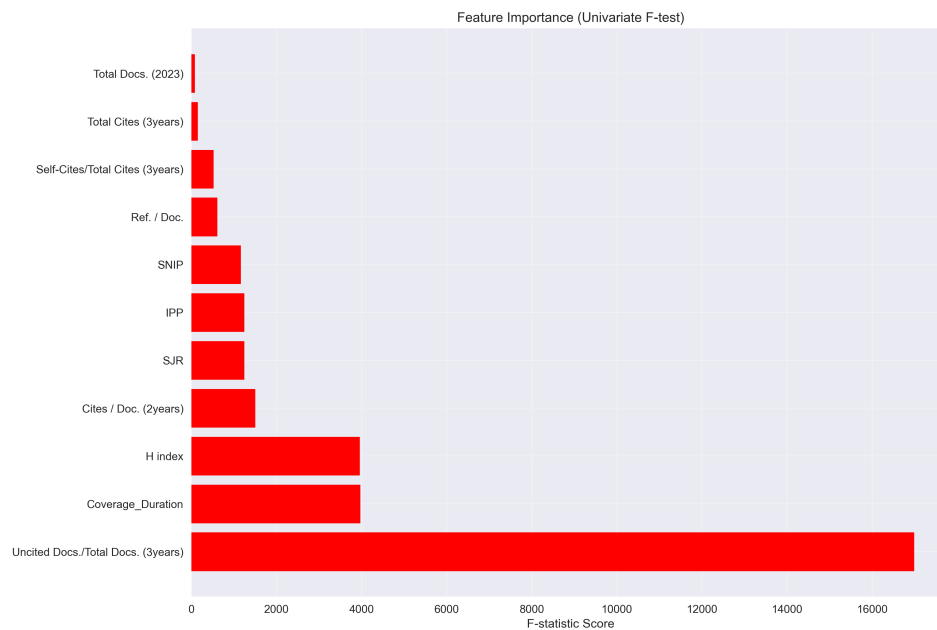


Figure 3.9.: This image shows the features that the best-performing model found most useful for telling apart problematic and legitimate journals. These are the most influential metrics when it comes to building a prediction system.

4. Classification Analysis

To understand which journals might be problematic, we built and tested several machine learning models. These models learn from past data—such as citation metrics, indexing scores, and citation behaviors—to predict whether a journal is likely to be legitimate or suspicious.

What We Did:

1. Input data: We used metrics like SJR, H-index, IPP, SNIP, and self-citation patterns for each journal.
2. Labels: Each journal was labeled as either legitimate (1) or problematic (0), based on external expert data.
3. Goal: Train models that can predict these labels from the journal metrics.

Once we had cleaned the data and selected meaningful features, we trained machine learning models to predict whether a journal is legitimate or problematic. Each model "learned" from patterns in the metrics of known journals.

Models Trained:

We trained five different classifiers:

1. Logistic Regression
2. Random Forest
3. Gradient Boosting
4. Support Vector Machine (SVM)
5. Naive Bayes

Model Training and Evaluation

The dataset was divided into two parts:

- **Training Set (80%)** – used to train the machine learning models.
- **Test Set (20%)** – used to evaluate model performance on unseen data.

To improve robustness, all features were standardized using the `RobustScaler`, which handles outliers better than traditional scaling methods.

Evaluation Metrics: Each model was assessed using the following metrics:

- **Accuracy** – The overall proportion of correct predictions.
- **Precision** – The proportion of journals predicted as problematic that were truly problematic.
- **Recall** – The proportion of actual problematic journals that were correctly identified.
- **F1-score** – The harmonic mean of precision and recall, useful for imbalanced datasets.
- **AUC (Area Under ROC Curve)** – Measures how well the model distinguishes between classes.
- **Cross-validation Accuracy** – The mean accuracy over 5-fold splits, showing model consistency.

Table 4.1.: Evaluation Metrics for Classification Models

Model	Accuracy	Precision	Recall	F1-Score	AUC	CV Accuracy $\pm$ SD
Logistic Regression	0.9543	0.9669	0.9679	0.9674	0.9769	0.9502 ± 0.0017
Random Forest	0.9461	0.9606	0.9626	0.9616	0.9805	0.9404 ± 0.0030
Gradient Boosting	0.9474	0.9597	0.9655	0.9626	0.9805	0.9418 ± 0.0024
SVM	0.9537	0.9658	0.9681	0.9670	0.9774	0.9484 ± 0.0018
Naive Bayes	0.7669	0.9760	0.6840	0.8043	0.9370	0.7577 ± 0.0098

Best Model Selection: Logistic Regression

Among all the models evaluated, **Logistic Regression** was selected as the best-performing model based on a combination of high predictive accuracy, interpretability, and stability. It achieved the following performance metrics on the test set:

- **Accuracy:** 95.43%

Logistic Regression was also preferred for its transparency and simplicity. Unlike ensemble models like Random Forest or Gradient Boosting, Logistic Regression provides clear coefficients that can be interpreted to understand the importance of each feature. This makes it particularly suitable for academic settings, where explainability and traceability of results are important.

5. Applying ML to the entire Dataset

The code for this chapter is located on GitHub at *Starma/scripts/collectingdata/4LabellingDataCWTS + Datasets.ipynb* in the "Train Test Model on the labeled Dataset" section.

This chapter will describe our one of the first ideas, namely the use of ML model on all labeled journals. The dataset was split into two parts: training and test. Because we have labeled data, i.e. the desired result, we need to use a supervised algorithm. We tried different ones, but we got the best result with Logistic Regression. The final score is 87 percent, which is a good result. As for precision and recall, they're shown in the following graph:

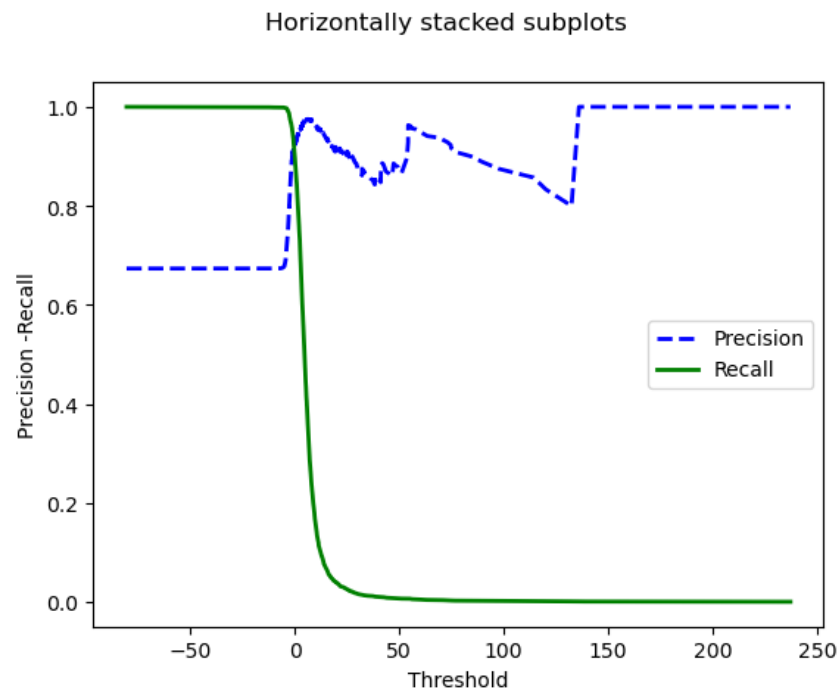


Figure 5.1.: Precision and recall versus the decision threshold

The receiver operating characteristic (ROC) curve is shown below and it is similar to the precision/recall curve, but instead of plotting precision versus recall, the ROC curve plots the recall against the false positive rate (ratio of negative instances that are incorrectly classified as positive). Ideally, the blue line should be as close to the upper left corner as possible, and our graph shows a good correlation.

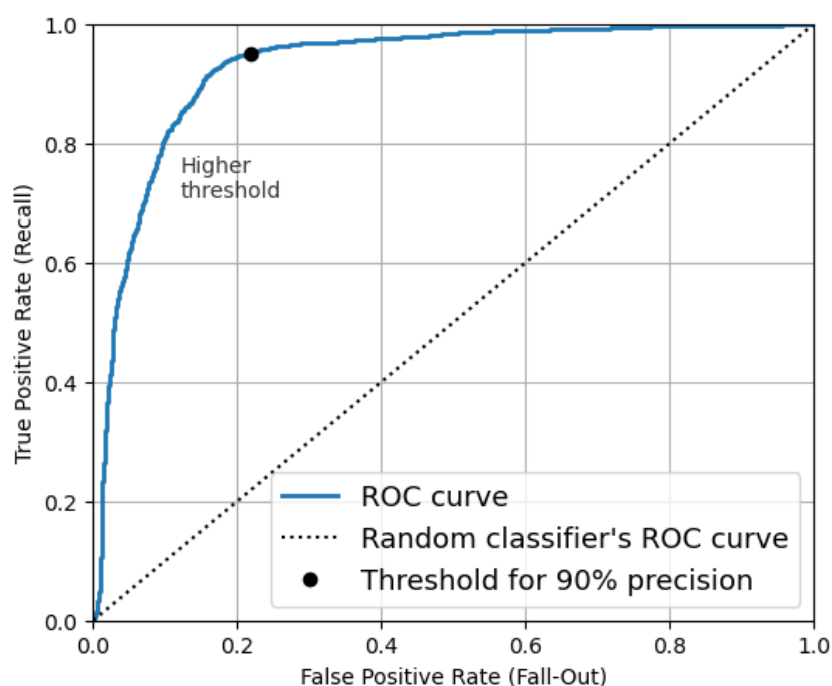


Figure 5.2.: The false positive rate against the true positive rate for all possible thresholds

Now let's analyze which features had what impact.

5. Applying ML to the entire Dataset

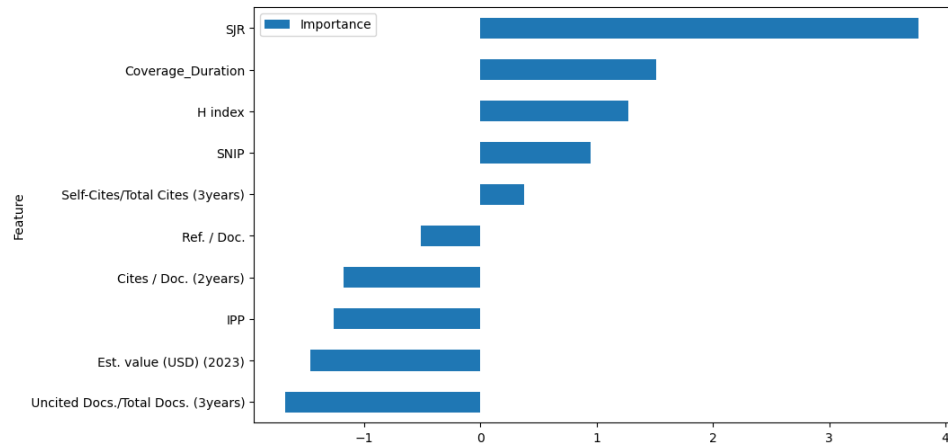


Figure 5.3.: Feature Importance

We assumed that the self-citation metric would play a major role in selecting poor journals, but the graph shows that it plays a minor role in a positive way. As expected, SJR and H index are good indicators of good journals. Uncited documents turned out to be the best metric for selecting poor journals, which was quite predictable.

The results obtained are quite good, but we wanted to get even better ones, so we assumed that each area might require its own approach, and therefore we decided to analyze several of them.

6. Machine Learning Results and Citation Behavior Analysis in Specific Areas

Mathematics Journals

Domain-Specific Analysis

Model Performance Comparison: First, three experiments were conducted to check different approaches for classifying Mathematics journals. First using the unbalanced dataset meaning the difference between good and bad journal is not balanced and is in different random order, it achieved an accuracy of 0.85; however, the performance was biased toward the dominant class, with an F1-score of 0.91 for Class 1 and only 0.65 for Class 0. After balancing the dataset with all its features intact, the model achieved its best and most reliable performance, reaching an accuracy of 0.92 and nearly equal F1-scores for both classes (0.93 for Class 0 and 0.92 for Class 1). When the balanced dataset was used with reduced features, the performance slightly declined, with an accuracy of 0.89 and F1-scores of 0.90 for Class 0 and 0.88 for Class 1. This suggests that some of the removed features contributed to the model's predictive capability.

This analysis examines citation behavior in specific domains such as Mathematics and whether bibliometric features can be of use to classify journals into high and low categories for predatory and trustworthiness. The objective is to understand how Mathematics journals behave in terms of citation patterns, and if features like SJR, H index, and the *uncited documents* ratio are reliable indicators of journal impact, this is particularly relevant because citation behavior in Mathematics differs significantly from that in fast-moving disciplines such as Medicine or Biology as it is shown in Figure 6.1.

6. Machine Learning Results and Citation Behavior Analysis in Specific Areas

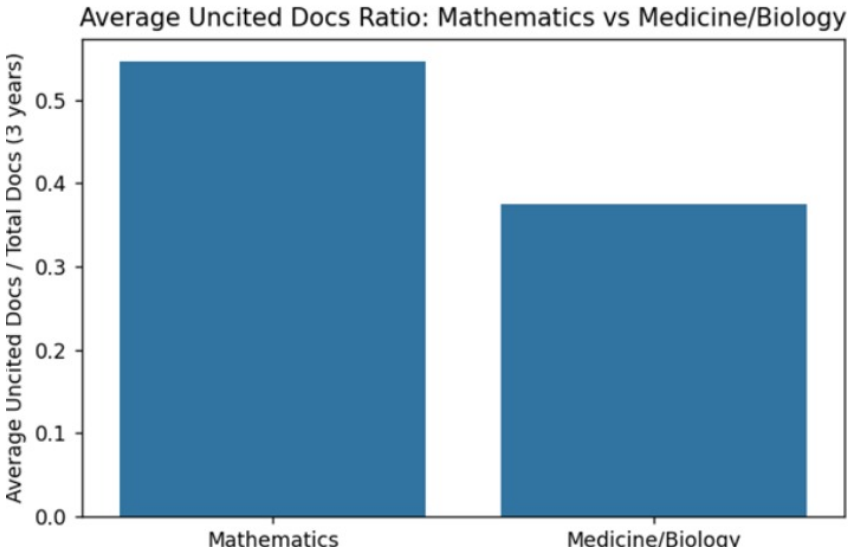


Figure 6.1.: Uncited docs ration

And so, uncited docs is one of the best options we have to use in the field of Mathematics, of which few landmark papers receive most citations, while the majority remain uncited. The data aligns with this expectation, as the average uncited ratio of 0.60 suggests that more than half of the papers go uncited even in established journals.

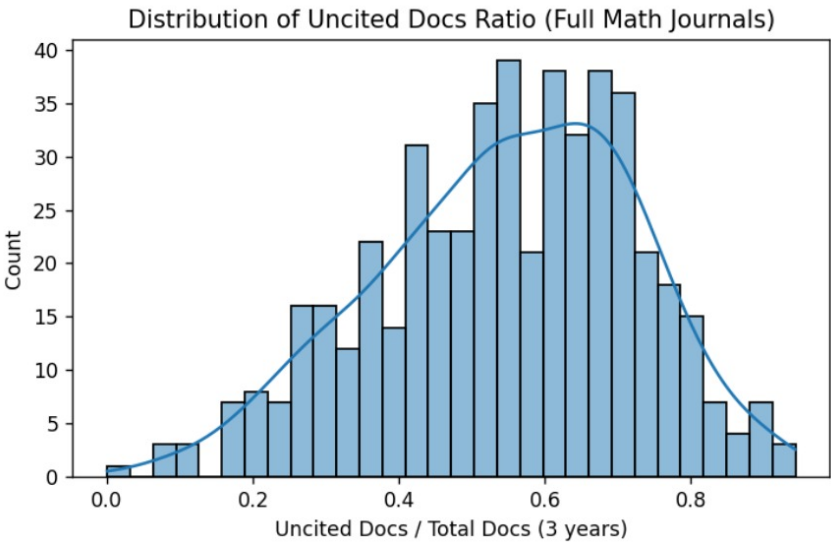


Figure 6.2.: Distribution of Uncited docs ration

The histogram of the Uncited Docs/Total Docs (3 years) ratio shows that most mathematics journals fall between 0.4 and 0.7, with a peak around 0.5 to 0.7, confirming that over half of published papers go uncited within three years.

Model Performance Comparison

Three experiments were conducted to evaluate different data processing approaches for classifying Mathematics journals. Using the unbalanced full dataset, the Random Forest classifier achieved an accuracy of 0.85; however, the performance was biased toward the dominant class, with an F1-score of 0.91 for Class 1 and only 0.65 for Class 0. After balancing the dataset while retaining all features, the model achieved its best and most reliable performance, reaching an accuracy of 0.92 and nearly equal F1-scores for both classes (0.93 for Class 0 and 0.92 for Class 1). When the balanced dataset was used with reduced features, the performance slightly declined, with an accuracy of 0.89 and F1-scores of 0.90 for Class 0 and 0.88 for Class 1. This suggests that some of the removed features contributed to the model's predictive capability.

Economics

We also analyzed the economics, but obtained worse results because the features did not correlate well with each other. The code with graphs can be found on GitHub at the path *Starma/scripts/analysis/Economy*.

7. Conclusion and Future Work

This project set out to identify signs of publication misconduct by analyzing over academic journals using clustering, classification, and statistical methods. The results were eye-opening:

1. About 30% of the journals showed signs of being potentially problematic.
2. Certain metrics (like self-citations and SJR scores) clearly stood out as strong indicators of misconduct.
3. Our best classification model (Logistic Regression) achieved over 95% accuracy, suggesting that machine learning can effectively flag suspicious journals.
4. The clustering analysis also revealed groups of journals and publishers behaving abnormally — some publishers only publish questionable journals.

These findings highlight the urgent need for better monitoring in academic publishing.

What's Next?

While our current work lays a solid foundation, there are still many areas to explore:

1. Using data sets covering a longer period of time(not only 2023) will enable a more accurate analysis of journals.
2. Expand to more datasets, including Scopus and DOAJ, to cross-check patterns.
3. Try more advanced models (like ensemble learning or LLM-based approaches) to improve detection further.
4. Drill down into citation networks to detect manipulation more precisely.
5. Automate regular audits of journals to maintain data freshness and reliability.

This project offers more than just an academic exercise — it's a step toward improving trust in scientific publishing. With scalable tools, transparent metrics, and collaboration across institutions, we can build a more honest and accountable research environment.

A. Source code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 # Set style for better visualizations
10 plt.style.use('seaborn-v0_8')
11 sns.set_palette("husl")
12
13 def load_and_prepare_data():
14     """Load and prepare the journal data for analysis"""
15     print("Loading journal data...")
16     df = pd.read_csv(r'C:\Users\TEJA\Downloads\tj\tj\
Data_with_Labels.csv')
17
18     # Display basic information about the dataset
19     print(f"Dataset shape: {df.shape}")
20     print(f"Number of legitimate journals (Label=1): {sum(df['
Labels'] == 1.0)}")
21     print(f"Number of potentially problematic journals (Label
=0): {sum(df['Labels'] == 0.0)}")
22
23     return df
24
25 def basic_descriptive_stats(df):
26     """Generate basic descriptive statistics"""
27     print("\n" + "="*60)
28     print("BASIC DESCRIPTIVE STATISTICS")
29     print("="*60)
30
31     # Select numerical columns for analysis
32     numerical_cols = ['SJR', 'H index', 'IPP', 'SNIP', 'Total
Docs. (2023)',
33                       'Total Docs. (3years)', 'Total Refs.', '
Est. value (USD) (2023)',
34                       'Total Cites (3years)', 'Self-Cites/Total
```

A. Source code

```
35     'Cites (3years)',
        'Uncited Docs./Total Docs. (3years)', '
36     Citable Docs. (3years)',
        'Cites / Doc. (2years)', 'Ref. / Doc.', '
    Coverage_Duration']

37
38     # Overall descriptive statistics
39     desc_stats = df[numerical_cols].describe()
40     print("\nOverall Descriptive Statistics:")
41     print(desc_stats.round(3))
42
43     # Save to CSV
44     desc_stats.to_csv('images/descriptive_statistics_overall.
    csv')
45
46     # Descriptive statistics by label
47     print("\nDescriptive Statistics by Journal Type:")
48     for label in [1.0, 0.0]:
49         label_name = "Legitimate" if label == 1.0 else "
    Potentially Problematic"
50         print(f"\n{label_name} Journals:")
51         subset = df[df['Labels'] == label][numerical_cols]
52         desc_subset = subset.describe()
53         print(desc_subset.round(3))
54
55         # Save to CSV
56         filename = f'images/descriptive_statistics_{label_name.
    lower().replace(" ", "_")}.csv'
57         desc_subset.to_csv(filename)
58
59     return desc_stats
60
61 def analyze_missing_data(df):
62     """Analyze missing data patterns"""
63     print("\n" + "="*60)
64     print("MISSING DATA ANALYSIS")
65     print("="*60)
66
67     missing_data = df.isnull().sum()
68     missing_percent = (missing_data / len(df)) * 100
69
70     missing_df = pd.DataFrame({
71         'Missing Count': missing_data,
72         'Missing Percentage': missing_percent
73     })
74
75     missing_df = missing_df[missing_df['Missing Count'] > 0].
    sort_values('Missing Count', ascending=False)
```

```

76
77     if len(missing_df) > 0:
78         print("Missing Data Summary:")
79         print(missing_df)
80         missing_df.to_csv('images/missing_data_analysis.csv')
81     else:
82         print("No missing data found in the dataset.")
83
84 def distribution_analysis(df):
85     """Analyze distributions of key variables"""
86     print("\n" + "="*60)
87     print("DISTRIBUTION ANALYSIS")
88     print("="*60)
89
90     # Key metrics for distribution analysis
91     key_metrics = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-Cites
92 /Total Cites (3years)',
93                   'Uncited Docs./Total Docs. (3years)', 'Cites
94 / Doc. (2years)']
95
96     # Create distribution plots
97     fig, axes = plt.subplots(3, 3, figsize=(15, 12))
98     axes = axes.ravel()
99
100    for i, metric in enumerate(key_metrics):
101        if i < len(axes):
102            # Histogram with KDE
103            axes[i].hist(df[metric], bins=50, alpha=0.7,
104            density=True, color='skyblue')
105
106            # Add KDE
107            try:
108                kde_data = df[metric].dropna()
109                if len(kde_data) > 1:
110                    kde = stats.gaussian_kde(kde_data)
111                    x_range = np.linspace(kde_data.min(),
112                    kde_data.max(), 100)
113                    axes[i].plot(x_range, kde(x_range), 'r-',
114                    linewidth=2)
115            except:
116                pass
117
118            axes[i].set_title(f'Distribution of {metric}')
119            axes[i].set_xlabel(metric)
120            axes[i].set_ylabel('Density')
121
122    # Remove empty subplots
123    for i in range(len(key_metrics), len(axes)):

```

A. Source code

```
119         fig.delaxes(axes[i])
120
121     plt.tight_layout()
122     plt.savefig('images/distribution_analysis.png', dpi=300,
123               bbox_inches='tight')
124     plt.close()
125
126     # Statistical tests for normality
127     print("\nNormality Tests (Shapiro-Wilk p-values):")
128     normality_results = {}
129     for metric in key_metrics:
130         data = df[metric].dropna()
131         if len(data) > 3: # Minimum sample size for Shapiro-
132             # Wilk
133             # Use a sample if data is too large
134             if len(data) > 5000:
135                 data = data.sample(5000, random_state=42)
136             stat, p_value = stats.shapiro(data)
137             normality_results[metric] = p_value
138             print(f"{metric}: p = {p_value:.6f}")
139
140     # Save normality test results
141     normality_df = pd.DataFrame(list(normality_results.items()),
142                                columns=['Metric', '
143                                Shapiro_Wilk_p_value'])
144     normality_df.to_csv('images/normality_tests.csv', index=
145                        False)
146
147 def correlation_analysis(df):
148     """Perform correlation analysis"""
149     print("\n" + "="*60)
150     print("CORRELATION ANALYSIS")
151     print("="*60)
152
153     # Select numerical columns for correlation
154     numerical_cols = ['SJR', 'H index', 'IPP', 'SNIP', 'Total
155                        Docs. (2023)',
156                        'Total Docs. (3years)', 'Total Refs.', '
157                        Total Cites (3years)',
158                        'Self-Cites/Total Cites (3years)', '
159                        Uncited Docs./Total Docs. (3years)',
160                        'Citable Docs. (3years)', 'Cites / Doc. (2
161                        years)', 'Ref. / Doc.',
162                        'Coverage_Duration', 'Labels']
163
164     # Calculate correlation matrix
165     corr_matrix = df[numerical_cols].corr()
```

```

158
159     # Create correlation heatmap
160     plt.figure(figsize=(14, 12))
161     mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
162     sns.heatmap(corr_matrix, mask=mask, annot=True, cmap='
coolwarm', center=0,
163                 square=True, linewidths=0.5, cbar_kws={"shrink"
: 0.8}, fmt='.2f')
164     plt.title('Correlation Matrix of Journal Metrics')
165     plt.tight_layout()
166     plt.savefig('images/correlation_matrix.png', dpi=300,
bbox_inches='tight')
167     plt.close()
168
169     # Save correlation matrix
170     corr_matrix.to_csv('images/correlation_matrix.csv')
171
172     # Find strongest correlations with Labels
173     label_correlations = corr_matrix['Labels'].abs().
sort_values(ascending=False)
174     print("\nStrongest correlations with journal legitimacy (
Labels):")
175     print(label_correlations.head(10))
176
177     return corr_matrix
178
179 def outlier_analysis(df):
180     """Identify and analyze outliers"""
181     print("\n" + "="*60)
182     print("OUTLIER ANALYSIS")
183     print("="*60)
184
185     key_metrics = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-Cites
/Total Cites (3years)',
186                  'Uncited Docs./Total Docs. (3years)']
187
188     outlier_summary = {}
189
190     for metric in key_metrics:
191         data = df[metric].dropna()
192         Q1 = data.quantile(0.25)
193         Q3 = data.quantile(0.75)
194         IQR = Q3 - Q1
195
196         lower_bound = Q1 - 1.5 * IQR
197         upper_bound = Q3 + 1.5 * IQR
198
199         outliers = data[(data < lower_bound) | (data >

```

A. Source code

```
upper_bound)]
200     outlier_count = len(outliers)
201     outlier_percentage = (outlier_count / len(data)) * 100
202
203     outlier_summary[metric] = {
204         'Count': outlier_count,
205         'Percentage': outlier_percentage,
206         'Lower_Bound': lower_bound,
207         'Upper_Bound': upper_bound
208     }
209
210     print(f"{metric}: {outlier_count} outliers ({
outlier_percentage:.2f}%)")
211
212     # Save outlier analysis
213     outlier_df = pd.DataFrame(outlier_summary).T
214     outlier_df.to_csv('images/outlier_analysis.csv')
215
216     return outlier_summary
217
218 def main():
219     """Main function to run all descriptive analyses"""
220     # Create images directory
221     import os
222     os.makedirs('images', exist_ok=True)
223
224     print("StAMA - Statistical Analysis in Modern Academia")
225     print("Script 1: Descriptive Statistics Analysis")
226     print("="*60)
227
228     # Load data
229     df = load_and_prepare_data()
230
231     # Run analyses
232     desc_stats = basic_descriptive_stats(df)
233     analyze_missing_data(df)
234     distribution_analysis(df)
235     corr_matrix = correlation_analysis(df)
236     outlier_summary = outlier_analysis(df)
237
238     print("\n" + "="*60)
239     print("DESCRIPTIVE ANALYSIS COMPLETE")
240     print("="*60)
241     print("Results saved in 'images' directory:")
242     print("- descriptive_statistics_*.csv")
243     print("- distribution_analysis.png")
244     print("- correlation_matrix.png and .csv")
245     print("- normality_tests.csv")
```

```
246     print("- outlier_analysis.csv")
247     print("- missing_data_analysis.csv (if applicable)")
248
249 if __name__ == "__main__":
250     main()
```

Listing A.1: Descriptive Statistics Analysis

A. Source code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from matplotlib.patches import Patch
6 import plotly.express as px
7 import plotly.graph_objects as go
8 from plotly.subplots import make_subplots
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 # Set style for better visualizations
13 plt.style.use('seaborn-v0_8')
14 sns.set_palette("husl")
15
16 def load_data():
17     """Load the journal data"""
18     print("Loading journal data for visualization...")
19     df = pd.read_csv(r'C:\Users\TEJA\Downloads\tj\tj\
20 Data_with_Labels.csv') # Correct
21
22     return df
23
24 def create_comparison_plots(df):
25     """Create comparison plots between legitimate and
26     problematic journals"""
27     print("Creating comparison visualizations...")
28
29     # Key metrics for comparison
30     key_metrics = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-Cites
31 /Total Cites (3years)',
32 'Uncited Docs./Total Docs. (3years)', 'Cites
33 / Doc. (2years)', 'Ref. / Doc.']
34
35     # Create box plots comparing legitimate vs problematic
36     journals
37     fig, axes = plt.subplots(2, 4, figsize=(20, 10))
38     axes = axes.ravel()
39
40     for i, metric in enumerate(key_metrics):
41         # Prepare data for plotting
42         legitimate = df[df['Labels'] == 1.0][metric].dropna()
43         problematic = df[df['Labels'] == 0.0][metric].dropna()
44
45         # Box plot
46         data_to_plot = [legitimate, problematic]
47         labels = ['Legitimate', 'Problematic']
```

```

44     box_plot = axes[i].boxplot(data_to_plot, labels=labels,
    patch_artist=True)
45
46     # Color the boxes
47     colors = ['lightblue', 'lightcoral']
48     for patch, color in zip(box_plot['boxes'], colors):
49         patch.set_facecolor(color)
50
51     axes[i].set_title(f'{metric}\nComparison by Journal
    Type')
52     axes[i].set_ylabel(metric)
53     axes[i].grid(True, alpha=0.3)
54
55     plt.tight_layout()
56     plt.savefig('images/journal_comparison_boxplots.png', dpi
    =300, bbox_inches='tight')
57     plt.close()
58
59 def create_distribution_comparisons(df):
60     """Create distribution comparison plots"""
61     print("Creating distribution comparison plots...")
62
63     key_metrics = ['SJR', 'H index', 'Self-Cites/Total Cites (3
    years)', 'Uncited Docs./Total Docs. (3years)']
64
65     fig, axes = plt.subplots(2, 2, figsize=(15, 10))
66     axes = axes.ravel()
67
68     for i, metric in enumerate(key_metrics):
69         legitimate = df[df['Labels'] == 1.0][metric].dropna()
70         problematic = df[df['Labels'] == 0.0][metric].dropna()
71
72         # Create histograms
73         axes[i].hist(legitimate, bins=50, alpha=0.7, label='
    Legitimate',
74                     color='lightblue', density=True)
75         axes[i].hist(problematic, bins=50, alpha=0.7, label='
    Problematic',
76                     color='lightcoral', density=True)
77
78         axes[i].set_title(f'Distribution of {metric}')
79         axes[i].set_xlabel(metric)
80         axes[i].set_ylabel('Density')
81         axes[i].legend()
82         axes[i].grid(True, alpha=0.3)
83
84     plt.tight_layout()
85     plt.savefig('images/distribution_comparisons.png', dpi=300,

```

A. Source code

```

    bbox_inches='tight')
86 plt.close()
87
88 def create_scatter_plots(df):
89     """Create scatter plots to identify patterns"""
90     print("Creating scatter plot analysis...")
91
92     # Key relationships to explore
93     relationships = [
94         ('SJR', 'H index'),
95         ('Self-Cites/Total Cites (3years)', 'SJR'),
96         ('Uncited Docs./Total Docs. (3years)', 'H index'),
97         ('IPP', 'SNIP')
98     ]
99
100     fig, axes = plt.subplots(2, 2, figsize=(15, 10))
101     axes = axes.ravel()
102
103     for i, (x_metric, y_metric) in enumerate(relationships):
104         # Separate data by label
105         legitimate = df[df['Labels'] == 1.0]
106         problematic = df[df['Labels'] == 0.0]
107
108         # Create scatter plots
109         axes[i].scatter(legitimate[x_metric], legitimate[
y_metric],
110                        alpha=0.6, c='blue', label='Legitimate',
s=20)
111         axes[i].scatter(problematic[x_metric], problematic[
y_metric],
112                        alpha=0.6, c='red', label='Problematic',
s=20)
113
114         axes[i].set_xlabel(x_metric)
115         axes[i].set_ylabel(y_metric)
116         axes[i].set_title(f'{y_metric} vs {x_metric}')
117         axes[i].legend()
118         axes[i].grid(True, alpha=0.3)
119
120     plt.tight_layout()
121     plt.savefig('images/scatter_plot_analysis.png', dpi=300,
bbox_inches='tight')
122     plt.close()
123
124 def create_publisher_analysis(df):
125     """Analyze patterns by publisher"""
126     print("Creating publisher analysis...")
127
```

```

128     # Count journals by publisher and label
129     publisher_counts = df.groupby(['Publisher', 'Labels']).size
130     ().unstack(fill_value=0)
131
132     # Calculate total journals per publisher
133     publisher_counts['Total'] = publisher_counts.sum(axis=1)
134
135     # Calculate percentage of problematic journals
136     if 0.0 in publisher_counts.columns:
137         publisher_counts['Problematic_Percentage'] = (
138             publisher_counts[0.0] / publisher_counts['Total']) * 100
139     else:
140         publisher_counts['Problematic_Percentage'] = 0
141
142     # Filter publishers with at least 5 journals
143     significant_publishers = publisher_counts[publisher_counts[
144         'Total'] >= 5]
145
146     # Sort by problematic percentage
147     significant_publishers = significant_publishers.sort_values
148     ('Problematic_Percentage', ascending=False)
149
150     # Create visualization
151     plt.figure(figsize=(15, 8))
152     top_publishers = significant_publishers.head(20)
153
154     bars = plt.bar(range(len(top_publishers)), top_publishers['
155         Problematic_Percentage'])
156     plt.xlabel('Publishers')
157     plt.ylabel('Percentage of Problematic Journals')
158     plt.title('Publishers with Highest Percentage of
159         Potentially Problematic Journals\n(Publishers with 5
160         journals)')
161     plt.xticks(range(len(top_publishers)), top_publishers.index
162         , rotation=45, ha='right')
163
164     # Color bars based on percentage
165     for i, bar in enumerate(bars):
166         percentage = top_publishers.iloc[i]['
167             Problematic_Percentage']
168         if percentage > 50:
169             bar.set_color('red')
170         elif percentage > 25:
171             bar.set_color('orange')
172         else:
173             bar.set_color('green')
174
175     plt.grid(True, alpha=0.3)

```

A. Source code

```
167     plt.tight_layout()
168     plt.savefig('images/publisher_analysis.png', dpi=300,
169               bbox_inches='tight')
170     plt.close()
171
172     # Save publisher analysis data
173     significant_publishers.to_csv('images/publisher_analysis.
174     csv')
175
176 def create_citation_patterns(df):
177     """Analyze citation patterns that might indicate misconduct
178     """
179     print("Analyzing citation patterns...")
180
181     # Create a comprehensive citation analysis plot
182     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
183
184     # 1. Self-citation rate distribution
185     axes[0, 0].hist(df[df['Labels'] == 1.0]['Self-Cites/Total
186     Cites (3years)'],
187                   bins=50, alpha=0.7, label='Legitimate',
188                   color='blue', density=True)
189     axes[0, 0].hist(df[df['Labels'] == 0.0]['Self-Cites/Total
190     Cites (3years)'],
191                   bins=50, alpha=0.7, label='Problematic',
192                   color='red', density=True)
193     axes[0, 0].set_title('Self-Citation Rate Distribution')
194     axes[0, 0].set_xlabel('Self-Cites/Total Cites')
195     axes[0, 0].legend()
196     axes[0, 0].grid(True, alpha=0.3)
197
198     # 2. Uncited documents ratio
199     axes[0, 1].hist(df[df['Labels'] == 1.0]['Uncited Docs./
200     Total Docs. (3years)'],
201                   bins=50, alpha=0.7, label='Legitimate',
202                   color='blue', density=True)
203     axes[0, 1].hist(df[df['Labels'] == 0.0]['Uncited Docs./
204     Total Docs. (3years)'],
205                   bins=50, alpha=0.7, label='Problematic',
206                   color='red', density=True)
207     axes[0, 1].set_title('Uncited Documents Ratio Distribution'
208     )
209     axes[0, 1].set_xlabel('Uncited Docs./Total Docs.')
210     axes[0, 1].legend()
211     axes[0, 1].grid(True, alpha=0.3)
212
213     # 3. Citations per document
214     axes[0, 2].hist(df[df['Labels'] == 1.0]['Cites / Doc. (2
```

```

years)'],
203         bins=50, alpha=0.7, label='Legitimate',
color='blue', density=True)
204 axes[0, 2].hist(df[df['Labels'] == 0.0]['Cites / Doc. (2
years)'],
205         bins=50, alpha=0.7, label='Problematic',
color='red', density=True)
206 axes[0, 2].set_title('Citations per Document Distribution')
207 axes[0, 2].set_xlabel('Cites / Doc. (2years)')
208 axes[0, 2].legend()
209 axes[0, 2].grid(True, alpha=0.3)
210
211 # 4. Self-citation vs SJR
212 legitimate = df[df['Labels'] == 1.0]
213 problematic = df[df['Labels'] == 0.0]
214
215 axes[1, 0].scatter(legitimate['Self-Cites/Total Cites (3
years)'], legitimate['SJR'],
216                 alpha=0.6, c='blue', label='Legitimate',
s=20)
217 axes[1, 0].scatter(problematic['Self-Cites/Total Cites (3
years)'], problematic['SJR'],
218                 alpha=0.6, c='red', label='Problematic',
s=20)
219 axes[1, 0].set_xlabel('Self-Cites/Total Cites')
220 axes[1, 0].set_ylabel('SJR')
221 axes[1, 0].set_title('Self-Citation Rate vs Journal Ranking
')
222 axes[1, 0].legend()
223 axes[1, 0].grid(True, alpha=0.3)
224
225 # 5. Uncited docs vs H-index
226 axes[1, 1].scatter(legitimate['Uncited Docs./Total Docs. (3
years)'], legitimate['H index'],
227                 alpha=0.6, c='blue', label='Legitimate',
s=20)
228 axes[1, 1].scatter(problematic['Uncited Docs./Total Docs.
(3years)'], problematic['H index'],
229                 alpha=0.6, c='red', label='Problematic',
s=20)
230 axes[1, 1].set_xlabel('Uncited Docs./Total Docs.')
231 axes[1, 1].set_ylabel('H index')
232 axes[1, 1].set_title('Uncited Documents vs H-index')
233 axes[1, 1].legend()
234 axes[1, 1].grid(True, alpha=0.3)
235
236 # 6. References per document distribution
237 axes[1, 2].hist(df[df['Labels'] == 1.0]['Ref. / Doc.'],

```

A. Source code

```
238         bins=50, alpha=0.7, label='Legitimate',
239         color='blue', density=True)
240     axes[1, 2].hist(df[df['Labels'] == 0.0]['Ref. / Doc.'],
241         bins=50, alpha=0.7, label='Problematic',
242         color='red', density=True)
243     axes[1, 2].set_title('References per Document Distribution')
244     axes[1, 2].set_xlabel('Ref. / Doc.')
245     axes[1, 2].legend()
246     axes[1, 2].grid(True, alpha=0.3)
247
248     plt.tight_layout()
249     plt.savefig('images/citation_patterns_analysis.png', dpi
250     =300, bbox_inches='tight')
251     plt.close()
252
253 def create_interactive_plots(df):
254     """Create interactive plots using Plotly"""
255     print("Creating interactive visualizations...")
256
257     # Interactive scatter plot
258     fig = px.scatter(df, x='SJR', y='H index', color='Labels',
259         hover_data=['Title', 'Publisher', 'Self-
260         Cites/Total Cites (3years)'],
261         title='Interactive Journal Analysis: SJR
262         vs H-index',
263         labels={'Labels': 'Journal Type'})
264
265     # Update color scale
266     fig.update_traces(marker=dict(size=8, opacity=0.7))
267     fig.write_html('images/interactive_scatter_plot.html')
268
269     # Interactive correlation heatmap
270     numerical_cols = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-
271         Cites/Total Cites (3years)',
272         'Uncited Docs./Total Docs. (3years)', '
273         Cites / Doc. (2years)', 'Labels']
274
275     corr_matrix = df[numerical_cols].corr()
276
277     fig_heatmap = go.Figure(data=go.Heatmap(
278         z=corr_matrix.values,
279         x=corr_matrix.columns,
280         y=corr_matrix.columns,
281         colorscale='RdBu',
282         zmid=0,
283         text=corr_matrix.round(3).values,
284         texttemplate="%{text}",
```

```

278         textfont={"size": 10},
279         hoverongaps=False))
280
281     fig_heatmap.update_layout(
282         title='Interactive Correlation Matrix',
283         xaxis_title='Variables',
284         yaxis_title='Variables'
285     )
286
287     fig_heatmap.write_html('images/
interactive_correlation_heatmap.html')
288
289 def main():
290     """Main function to run all visualization analyses"""
291     # Create images directory
292     import os
293     os.makedirs('images', exist_ok=True)
294
295     print("StAMA - Statistical Analysis in Modern Academia")
296     print("Script 2: Data Visualization")
297     print("="*60)
298
299     # Load data
300     df = load_data()
301
302     # Create visualizations
303     create_comparison_plots(df)
304     create_distribution_comparisons(df)
305     create_scatter_plots(df)
306     create_publisher_analysis(df)
307     create_citation_patterns(df)
308     create_interactive_plots(df)
309
310     print("\n" + "="*60)
311     print("VISUALIZATION ANALYSIS COMPLETE")
312     print("="*60)
313     print("Visualizations saved in 'images' directory:")
314     print("- journal_comparison_boxplots.png")
315     print("- distribution_comparisons.png")
316     print("- scatter_plot_analysis.png")
317     print("- publisher_analysis.png and .csv")
318     print("- citation_patterns_analysis.png")
319     print("- interactive_scatter_plot.html")
320     print("- interactive_correlation_heatmap.html")
321
322 if __name__ == "__main__":

```

A. Source code

```
323 main()
```

Listing A.2: Data Visualization

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 from scipy.stats import mannwhitneyu, chi2_contingency,
    pearsonr, spearmanr
7 import warnings
8 warnings.filterwarnings('ignore')
9
10 # Set style for better visualizations
11 plt.style.use('seaborn-v0_8')
12 sns.set_palette("husl")
13
14 def load_data():
15     """Load the journal data"""
16     print("Loading journal data for hypothesis testing...")
17     df = pd.read_csv(r'C:\Users\TEJA\Downloads\tj\tj\
    Data_with_Labels.csv') # Correct
18
19     return df
20
21 def test_normality(data, name):
22     """Test normality of data using multiple tests"""
23     results = {}
24
25     # Shapiro-Wilk test (for smaller samples)
26     if len(data) <= 5000:
27         stat, p_value = stats.shapiro(data)
28         results['Shapiro-Wilk'] = {'statistic': stat, 'p_value'
    : p_value}
29
30     # Kolmogorov-Smirnov test
31     stat, p_value = stats.kstest(data, 'norm', args=(data.mean
    (), data.std()))
32     results['Kolmogorov-Smirnov'] = {'statistic': stat, '
    p_value': p_value}
33
34     # Anderson-Darling test
35     result = stats.anderson(data, dist='norm')
36     results['Anderson-Darling'] = {'statistic': result.
    statistic, 'critical_values': result.critical_values}
37
38     return results
39
40 def perform_two_sample_tests(df):
41     """Perform two-sample tests comparing legitimate vs
    problematic journals"""

```

A. Source code

```
42     print("\n" + "="*60)
43     print("TWO-SAMPLE HYPOTHESIS TESTS")
44     print("="*60)
45
46     # Key metrics to test
47     test_metrics = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-
48     Cites/Total Cites (3years)',
49     'Uncited Docs./Total Docs. (3years)', 'Cites
50     / Doc. (2years)', 'Ref. / Doc.',
51     'Total Docs. (2023)', 'Total Cites (3years)'
52     , 'Coverage_Duration']
53
54     results = []
55
56     for metric in test_metrics:
57         print(f"\nTesting: {metric}")
58         print("-" * 40)
59
60         # Separate data by label
61         legitimate = df[df['Labels'] == 1.0][metric].dropna()
62         problematic = df[df['Labels'] == 0.0][metric].dropna()
63
64         if len(legitimate) == 0 or len(problematic) == 0:
65             print(f"Insufficient data for {metric}")
66             continue
67
68         # Basic statistics
69         leg_mean, leg_std = legitimate.mean(), legitimate.std()
70         prob_mean, prob_std = problematic.mean(), problematic.
71         std()
72
73         print(f"Legitimate journals: n={len(legitimate)}, mean
74         ={leg_mean:.4f}, std={leg_std:.4f}")
75         print(f"Problematic journals: n={len(problematic)},
76         mean={prob_mean:.4f}, std={prob_std:.4f}")
77
78         # Test normality
79         leg_normality = test_normality(legitimate, f"{metric}
80         _legitimate")
81         prob_normality = test_normality(problematic, f"{metric}
82         _problematic")
83
84         # Check if data is approximately normal (using Shapiro-
85         Wilk p-value > 0.05)
86         leg_normal = leg_normality.get('Shapiro-Wilk', {}).get(
87         'p_value', 0) > 0.05 if len(legitimate) <= 5000 else False
88         prob_normal = prob_normality.get('Shapiro-Wilk', {}).
89         get('p_value', 0) > 0.05 if len(problematic) <= 5000 else
```

False

```
79
80     # Perform appropriate tests
81     test_results = {}
82
83     # 1. Mann-Whitney U test (non-parametric)
84     u_stat, u_p = mannwhitneyu(legitimate, problematic,
alternative='two-sided')
85     test_results['Mann-Whitney U'] = {'statistic': u_stat,
'p_value': u_p}
86     print(f"Mann-Whitney U test: U={u_stat:.4f}, p={u_p:.6f
}")
87
88     # 2. Welch's t-test (doesn't assume equal variances)
89     t_stat, t_p = stats.ttest_ind(legitimate, problematic,
equal_var=False)
90     test_results['Welch t-test'] = {'statistic': t_stat, '
p_value': t_p}
91     print(f"Welch's t-test: t={t_stat:.4f}, p={t_p:.6f}")
92
93     # 3. Levene's test for equal variances
94     lev_stat, lev_p = stats.levene(legitimate, problematic)
95     test_results['Levene (equal var)'] = {'statistic':
lev_stat, 'p_value': lev_p}
96     print(f"Levene's test (equal variances): F={lev_stat:.4
f}, p={lev_p:.6f}")
97
98     # 4. Effect size (Cohen's d)
99     pooled_std = np.sqrt(((len(legitimate) - 1) * leg_std
**2 + (len(problematic) - 1) * prob_std**2) /
100                        (len(legitimate) + len(problematic)
- 2))
101     cohens_d = (leg_mean - prob_mean) / pooled_std
102     test_results['Cohen_d'] = cohens_d
103     print(f"Cohen's d (effect size): {cohens_d:.4f}")
104
105     # Interpret effect size
106     if abs(cohens_d) < 0.2:
107         effect_size = "Small"
108     elif abs(cohens_d) < 0.5:
109         effect_size = "Medium"
110     elif abs(cohens_d) < 0.8:
111         effect_size = "Large"
112     else:
113         effect_size = "Very Large"
114
115     print(f"Effect size interpretation: {effect_size}")
116
```

A. Source code

```
117         # Store results
118         result_row = {
119             'Metric': metric,
120             'Legitimate_n': len(legitimate),
121             'Legitimate_mean': leg_mean,
122             'Legitimate_std': leg_std,
123             'Problematic_n': len(problematic),
124             'Problematic_mean': prob_mean,
125             'Problematic_std': prob_std,
126             'Mann_Whitney_U': u_stat,
127             'Mann_Whitney_p': u_p,
128             'Welch_t': t_stat,
129             'Welch_t_p': t_p,
130             'Levene_F': lev_stat,
131             'Levene_p': lev_p,
132             'Cohens_d': cohens_d,
133             'Effect_size': effect_size
134         }
135
136         results.append(result_row)
137
138     # Convert to DataFrame and save
139     results_df = pd.DataFrame(results)
140     results_df.to_csv('images/hypothesis_test_results.csv',
141                      index=False)
142
143     return results_df
144
145 def confidence_intervals_analysis(df):
146     """Calculate confidence intervals for key metrics"""
147     print("\n" + "="*60)
148     print("CONFIDENCE INTERVALS ANALYSIS")
149     print("="*60)
150
151     key_metrics = ['SJR', 'H index', 'Self-Cites/Total Cites (3
152                    years)',
153                    'Uncited Docs./Total Docs. (3years)', 'Cites
154                    / Doc. (2years)']
155
156     confidence_level = 0.95
157     alpha = 1 - confidence_level
158
159     ci_results = []
160
161     for metric in key_metrics:
162         print(f"\nConfidence Intervals for {metric}:")
163         print("-" * 40)
```

```

162         for label, label_name in [(1.0, 'Legitimate'), (0.0, '
163             Problematic')]:
164                 data = df[df['Labels'] == label][metric].dropna()
165
166                 if len(data) == 0:
167                     continue
168
169                 n = len(data)
170                 mean = data.mean()
171                 std = data.std()
172                 se = std / np.sqrt(n)
173
174                 # t-distribution critical value
175                 t_critical = stats.t.ppf(1 - alpha/2, df=n-1)
176
177                 # Confidence interval
178                 ci_lower = mean - t_critical * se
179                 ci_upper = mean + t_critical * se
180
181                 print(f"{label_name}: n={n}, mean={mean:.4f}")
182                 print(f"95% CI: [{ci_lower:.4f}, {ci_upper:.4f}]")
183
184                 # Bootstrap confidence interval
185                 bootstrap_means = []
186                 n_bootstrap = 1000
187
188                 for _ in range(n_bootstrap):
189                     bootstrap_sample = np.random.choice(data, size=
190                         n, replace=True)
191                     bootstrap_means.append(np.mean(bootstrap_sample
192                         ))
193
194                 bootstrap_ci_lower = np.percentile(bootstrap_means,
195                     2.5)
196                 bootstrap_ci_upper = np.percentile(bootstrap_means,
197                     97.5)
198
199                 print(f"Bootstrap 95% CI: [{bootstrap_ci_lower:.4f
200                     }, {bootstrap_ci_upper:.4f}]")
201
202                 ci_results.append({
203                     'Metric': metric,
204                     'Journal_Type': label_name,
205                     'n': n,
206                     'Mean': mean,
207                     'Std': std,
208                     'SE': se,
209                     'CI_Lower': ci_lower,

```

A. Source code

```
204         'CI_Upper': ci_upper,
205         'Bootstrap_CI_Lower': bootstrap_ci_lower,
206         'Bootstrap_CI_Upper': bootstrap_ci_upper
207     })
208
209     # Save confidence intervals
210     ci_df = pd.DataFrame(ci_results)
211     ci_df.to_csv('images/confidence_intervals.csv', index=False)
212
213     return ci_df
214
215 def correlation_hypothesis_tests(df):
216     """Test correlations between variables and journal
217     legitimacy"""
218     print("\n" + "="*60)
219     print("CORRELATION HYPOTHESIS TESTS")
220     print("="*60)
221
222     numerical_cols = ['SJR', 'H index', 'IPP', 'SNIP', 'Self-
223     Cites/Total Cites (3years)',
224                       'Uncited Docs./Total Docs. (3years)', '
225     Cites / Doc. (2years)', 'Ref. / Doc.',
226                       'Total Docs. (2023)', 'Total Cites (3years
227     )', 'Coverage_Duration']
228
229     correlation_results = []
230
231     for col in numerical_cols:
232         data_col = df[col].dropna()
233         labels_col = df.loc[data_col.index, 'Labels']
234
235         # Pearson correlation
236         pearson_r, pearson_p = pearsonr(data_col, labels_col)
237
238         # Spearman correlation
239         spearman_r, spearman_p = spearmanr(data_col, labels_col)
240
241         print(f"\n{col}:")
242         print(f"Pearson r = {pearson_r:.4f}, p = {pearson_p:.6f
243         }")
244         print(f"Spearman rho = {spearman_r:.4f}, p = {
245         spearman_p:.6f}")
246
247         correlation_results.append({
248             'Variable': col,
249             'Pearson_r': pearson_r,
```

```

244         'Pearson_p': pearson_p,
245         'Spearman_rho': spearman_r,
246         'Spearman_p': spearman_p
247     })
248
249     # Save correlation results
250     corr_df = pd.DataFrame(correlation_results)
251     corr_df.to_csv('images/correlation_hypothesis_tests.csv',
252                   index=False)
253
254     return corr_df
255
256 def multiple_testing_correction(results_df):
257     """Apply multiple testing corrections"""
258     print("\n" + "="*60)
259     print("MULTIPLE TESTING CORRECTION")
260     print("="*60)
261
262     # Extract p-values for correction
263     p_values = results_df['Mann_Whitney_p'].values
264
265     # Bonferroni correction
266     bonferroni_alpha = 0.05 / len(p_values)
267     bonferroni_significant = p_values < bonferroni_alpha
268
269     # Benjamini-Hochberg (FDR) correction
270     from statsmodels.stats.multitest import multipletests
271     fdr_reject, fdr_pvals, _, _ = multipletests(p_values, alpha
272 =0.05, method='fdr_bh')
273
274     # Add corrections to results
275     results_df['Bonferroni_alpha'] = bonferroni_alpha
276     results_df['Bonferroni_significant'] =
277     bonferroni_significant
278     results_df['FDR_corrected_p'] = fdr_pvals
279     results_df['FDR_significant'] = fdr_reject
280
281     print(f"Original alpha level: 0.05")
282     print(f"Bonferroni corrected alpha: {bonferroni_alpha:.6f}"
283 )
284     print(f"Number of tests: {len(p_values)}")
285     print(f"Significant after Bonferroni: {sum(
286     bonferroni_significant)}")
287     print(f"Significant after FDR: {sum(fdr_reject)}")
288
289     # Save corrected results
290     results_df.to_csv('images/hypothesis_test_results_corrected
291 .csv', index=False)

```

A. Source code

```
286     return results_df
287
288
289 def create_hypothesis_visualizations(results_df, ci_df):
290     """Create visualizations for hypothesis test results"""
291     print("\nCreating hypothesis test visualizations...")
292
293     # 1. Effect sizes plot
294     plt.figure(figsize=(12, 8))
295     metrics = results_df['Metric']
296     effect_sizes = results_df['Cohens_d']
297     colors = ['red' if abs(d) >= 0.8 else 'orange' if abs(d) >=
298              0.5 else 'yellow' if abs(d) >= 0.2 else 'green'
299              for d in effect_sizes]
300
301     bars = plt.barh(range(len(metrics)), effect_sizes, color=
302                     colors)
303     plt.yticks(range(len(metrics)), metrics)
304     plt.xlabel("Cohen's d (Effect Size)")
305     plt.title("Effect Sizes: Legitimate vs Problematic Journals")
306
307     plt.axvline(x=0, color='black', linestyle='-', alpha=0.3)
308     plt.axvline(x=0.2, color='gray', linestyle='--', alpha=0.5,
309                 label='Small effect')
310     plt.axvline(x=0.5, color='gray', linestyle='--', alpha=0.5,
311                 label='Medium effect')
312     plt.axvline(x=0.8, color='gray', linestyle='--', alpha=0.5,
313                 label='Large effect')
314     plt.axvline(x=-0.2, color='gray', linestyle='--', alpha
315                 =0.5)
316     plt.axvline(x=-0.5, color='gray', linestyle='--', alpha
317                 =0.5)
318     plt.axvline(x=-0.8, color='gray', linestyle='--', alpha
319                 =0.5)
320     plt.legend()
321     plt.grid(True, alpha=0.3)
322     plt.tight_layout()
323     plt.savefig('images/effect_sizes_plot.png', dpi=300,
324                 bbox_inches='tight')
325     plt.close()
326
327     # 2. P-values plot
328     plt.figure(figsize=(12, 8))
329     p_values = results_df['Mann_Whitney_p']
330     significant = p_values < 0.05
331     colors = ['red' if sig else 'blue' for sig in significant]
332
333     bars = plt.barh(range(len(metrics)), -np.log10(p_values),
```

```

color=colors)
324 plt.yticks(range(len(metrics)), metrics)
325 plt.xlabel("-log10(p-value)")
326 plt.title("Statistical Significance: Mann-Whitney U Tests")
327 plt.axvline(x=-np.log10(0.05), color='red', linestyle='--',
label=' = 0.05')
328 plt.axvline(x=-np.log10(0.01), color='orange', linestyle='
--', label=' = 0.01')
329 plt.legend()
330 plt.grid(True, alpha=0.3)
331 plt.tight_layout()
332 plt.savefig('images/p_values_plot.png', dpi=300,
bbox_inches='tight')
333 plt.close()
334
335 def main():
336     """Main function to run all hypothesis testing analyses"""
337     # Create images directory
338     import os
339     os.makedirs('images', exist_ok=True)
340
341     print("StAMA - Statistical Analysis in Modern Academia")
342     print("Script 3: Hypothesis Testing and Confidence
Intervals")
343     print("="*60)
344
345     # Load data
346     df = load_data()
347
348     # Perform analyses
349     results_df = perform_two_sample_tests(df)
350     ci_df = confidence_intervals_analysis(df)
351     corr_df = correlation_hypothesis_tests(df)
352     corrected_results_df = multiple_testing_correction(
results_df)
353     create_hypothesis_visualizations(corrected_results_df,
ci_df)
354
355     print("\n" + "="*60)
356     print("HYPOTHESIS TESTING ANALYSIS COMPLETE")
357     print("="*60)
358     print("Results saved in 'images' directory:")
359     print("- hypothesis_test_results.csv")
360     print("- hypothesis_test_results_corrected.csv")
361     print("- confidence_intervals.csv")
362     print("- correlation_hypothesis_tests.csv")
363     print("- effect_sizes_plot.png")
364     print("- p_values_plot.png")

```

A. Source code

```
365  
366 if __name__ == "__main__":  
367     main()
```

Listing A.3: Hypothesis Testing and Confidence Intervals

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.cluster import KMeans, DBSCAN,
    AgglomerativeClustering
6 from sklearn.preprocessing import StandardScaler, RobustScaler
7 from sklearn.decomposition import PCA
8 from sklearn.metrics import silhouette_score,
    adjusted_rand_score, calinski_harabasz_score
9 from sklearn.manifold import TSNE
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # Set style for better visualizations
14 plt.style.use('seaborn-v0_8')
15 sns.set_palette("husl")
16
17 def load_and_prepare_data():
18     """Load and prepare data for clustering analysis"""
19     print("Loading and preparing data for clustering analysis
    ...")
20     df = pd.read_csv(r'C:\Users\TEJA\Downloads\tj\tj\
    Data_with_Labels.csv') # Correct
21
22
23     # Select features for clustering
24     clustering_features = [
25         'SJR', 'H index', 'IPP', 'SNIP',
26         'Self-Cites/Total Cites (3years)',
27         'Uncited Docs./Total Docs. (3years)',
28         'Cites / Doc. (2years)',
29         'Ref. / Doc.',
30         'Total Docs. (2023)',
31         'Total Cites (3years)',
32         'Coverage_Duration'
33     ]
34
35     # Create feature matrix
36     X = df[clustering_features].copy()
37
38     # Handle missing values
39     X = X.fillna(X.median())
40
41     # Remove extreme outliers (beyond 3 standard deviations)
42     for col in X.columns:
43         mean = X[col].mean()
44         std = X[col].std()

```

A. Source code

```
45     X = X[(X[col] >= mean - 3*std) & (X[col] <= mean + 3*
46         std)]
47
48     # Get corresponding labels and titles
49     y_true = df.loc[X.index, 'Labels'].values
50     titles = df.loc[X.index, 'Title'].values
51     publishers = df.loc[X.index, 'Publisher'].values
52
53     print(f"Final dataset shape: {X.shape}")
54     print(f"Features used: {clustering_features}")
55
56     return X, y_true, titles, publishers, clustering_features
57
58 def scale_features(X):
59     """Scale features using different methods"""
60     print("Scaling features...")
61
62     # Standard scaling
63     scaler_standard = StandardScaler()
64     X_standard = scaler_standard.fit_transform(X)
65
66     # Robust scaling (less sensitive to outliers)
67     scaler_robust = RobustScaler()
68     X_robust = scaler_robust.fit_transform(X)
69
70     return X_standard, X_robust, scaler_standard, scaler_robust
71
72 def determine_optimal_clusters(X_scaled, max_clusters=15):
73     """Determine optimal number of clusters using multiple
74     methods"""
75     print("Determining optimal number of clusters...")
76
77     # Elbow method
78     inertias = []
79     silhouette_scores = []
80     calinski_scores = []
81
82     K_range = range(2, max_clusters + 1)
83
84     for k in K_range:
85         kmeans = KMeans(n_clusters=k, random_state=42, n_init
86             =10)
87         cluster_labels = kmeans.fit_predict(X_scaled)
88
89         inertias.append(kmeans.inertia_)
90         silhouette_scores.append(silhouette_score(X_scaled,
91             cluster_labels))
92         calinski_scores.append(calinski_harabasz_score(X_scaled
```

```

, cluster_labels))

89
90 # Create plots
91 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
92
93 # Elbow plot
94 axes[0].plot(K_range, inertias, 'bo-')
95 axes[0].set_xlabel('Number of Clusters (k)')
96 axes[0].set_ylabel('Inertia')
97 axes[0].set_title('Elbow Method for Optimal k')
98 axes[0].grid(True, alpha=0.3)
99
100 # Silhouette score plot
101 axes[1].plot(K_range, silhouette_scores, 'ro-')
102 axes[1].set_xlabel('Number of Clusters (k)')
103 axes[1].set_ylabel('Silhouette Score')
104 axes[1].set_title('Silhouette Score vs Number of Clusters')
105 axes[1].grid(True, alpha=0.3)
106
107 # Calinski-Harabasz score plot
108 axes[2].plot(K_range, calinski_scores, 'go-')
109 axes[2].set_xlabel('Number of Clusters (k)')
110 axes[2].set_ylabel('Calinski-Harabasz Score')
111 axes[2].set_title('Calinski-Harabasz Score vs Number of
Clusters')
112 axes[2].grid(True, alpha=0.3)
113
114 plt.tight_layout()
115 plt.savefig('images/optimal_clusters_analysis.png', dpi
=300, bbox_inches='tight')
116 plt.close()
117
118 # Find optimal k based on silhouette score
119 optimal_k = K_range[np.argmax(silhouette_scores)]
120
121 print(f"Optimal number of clusters based on silhouette
score: {optimal_k}")
122 print(f"Best silhouette score: {max(silhouette_scores):.4f}
")
123
124 # Save results
125 results_df = pd.DataFrame({
126     'k': K_range,
127     'inertia': inertias,
128     'silhouette_score': silhouette_scores,
129     'calinski_harabasz_score': calinski_scores
130 })
131 results_df.to_csv('images/cluster_optimization_results.csv')

```

A. Source code

```
, index=False)
132
133     return optimal_k, results_df
134
135 def perform_kmeans_clustering(X_scaled, optimal_k, y_true):
136     """Perform K-means clustering"""
137     print(f"Performing K-means clustering with k={optimal_k}...")
138
139     kmeans = KMeans(n_clusters=optimal_k, random_state=42,
140 n_init=10)
141     cluster_labels = kmeans.fit_predict(X_scaled)
142
143     # Calculate metrics
144     silhouette_avg = silhouette_score(X_scaled, cluster_labels)
145     calinski_score = calinski_harabasz_score(X_scaled,
146 cluster_labels)
147     ari_score = adjusted_rand_score(y_true, cluster_labels)
148
149     print(f"K-means Results:")
150     print(f"Silhouette Score: {silhouette_avg:.4f}")
151     print(f"Calinski-Harabasz Score: {calinski_score:.4f}")
152     print(f"Adjusted Rand Index: {ari_score:.4f}")
153
154     return kmeans, cluster_labels, silhouette_avg
155
156 def perform_dbscan_clustering(X_scaled, y_true):
157     """Perform DBSCAN clustering"""
158     print("Performing DBSCAN clustering...")
159
160     # Try different eps values
161     eps_values = [0.3, 0.5, 0.7, 1.0, 1.5]
162     best_eps = None
163     best_score = -1
164     best_labels = None
165
166     for eps in eps_values:
167         dbscan = DBSCAN(eps=eps, min_samples=5)
168         labels = dbscan.fit_predict(X_scaled)
169
170         # Skip if all points are noise or all in one cluster
171         n_clusters = len(set(labels)) - (1 if -1 in labels else
172 0)
173         if n_clusters < 2:
174             continue
175
176         # Calculate silhouette score (excluding noise points)
177         if len(set(labels)) > 1:
```

```

175         mask = labels != -1
176         if np.sum(mask) > 1:
177             score = silhouette_score(X_scaled[mask], labels
[178 mask]))
179             if score > best_score:
180                 best_score = score
181                 best_eps = eps
182                 best_labels = labels
183
184         if best_labels is not None:
185             n_clusters = len(set(best_labels)) - (1 if -1 in
best_labels else 0)
186             n_noise = list(best_labels).count(-1)
187             ari_score = adjusted_rand_score(y_true, best_labels)
188
189             print(f"DBSCAN Results (eps={best_eps}):")
190             print(f"Number of clusters: {n_clusters}")
191             print(f"Number of noise points: {n_noise}")
192             print(f"Silhouette Score: {best_score:.4f}")
193             print(f"Adjusted Rand Index: {ari_score:.4f}")
194         else:
195             print("DBSCAN: No suitable clustering found")
196             best_labels = np.zeros(len(X_scaled))
197
198         return best_labels, best_eps
199
200 def perform_hierarchical_clustering(X_scaled, optimal_k, y_true
):
201     """Perform hierarchical clustering"""
202     print(f"Performing hierarchical clustering with k={
optimal_k}...")
203
204     hierarchical = AgglomerativeClustering(n_clusters=optimal_k
, linkage='ward')
205     cluster_labels = hierarchical.fit_predict(X_scaled)
206
207     # Calculate metrics
208     silhouette_avg = silhouette_score(X_scaled, cluster_labels)
209     calinski_score = calinski_harabasz_score(X_scaled,
cluster_labels)
210     ari_score = adjusted_rand_score(y_true, cluster_labels)
211
212     print(f"Hierarchical Clustering Results:")
213     print(f"Silhouette Score: {silhouette_avg:.4f}")
214     print(f"Calinski-Harabasz Score: {calinski_score:.4f}")
215     print(f"Adjusted Rand Index: {ari_score:.4f}")
216
217     return hierarchical, cluster_labels

```

A. Source code

```
217
218 def visualize_clusters_pca(X_scaled, cluster_labels, y_true,
    method_name):
219     """Visualize clusters using PCA"""
220     print(f"Creating PCA visualization for {method_name}...")
221
222     # Perform PCA
223     pca = PCA(n_components=2)
224     X_pca = pca.fit_transform(X_scaled)
225
226     # Create subplots
227     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
228
229     # Plot clusters
230     scatter1 = axes[0].scatter(X_pca[:, 0], X_pca[:, 1], c=
    cluster_labels, cmap='viridis', alpha=0.7)
231     axes[0].set_xlabel(f'First Principal Component ({pca.
    explained_variance_ratio_[0]:.2%} variance)')
232     axes[0].set_ylabel(f'Second Principal Component ({pca.
    explained_variance_ratio_[1]:.2%} variance)')
233     axes[0].set_title(f'{method_name} Clusters (PCA)')
234     plt.colorbar(scatter1, ax=axes[0])
235
236     # Plot true labels
237     scatter2 = axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=
    y_true, cmap='RdYlBu', alpha=0.7)
238     axes[1].set_xlabel(f'First Principal Component ({pca.
    explained_variance_ratio_[0]:.2%} variance)')
239     axes[1].set_ylabel(f'Second Principal Component ({pca.
    explained_variance_ratio_[1]:.2%} variance)')
240     axes[1].set_title('True Labels (PCA)')
241     plt.colorbar(scatter2, ax=axes[1])
242
243     plt.tight_layout()
244     plt.savefig(f'images/{method_name.lower().replace(" ", "_")
    }_pca_visualization.png',
245               dpi=300, bbox_inches='tight')
246     plt.close()
247
248     return pca, X_pca
249
250 def visualize_clusters_tsne(X_scaled, cluster_labels, y_true,
    method_name):
251     """Visualize clusters using t-SNE"""
252     print(f"Creating t-SNE visualization for {method_name}...")
253
254     # Perform t-SNE (use a subset if data is too large)
255     if len(X_scaled) > 1000:
```

```

256         indices = np.random.choice(len(X_scaled), 1000, replace
= False)
257         X_subset = X_scaled[indices]
258         cluster_subset = cluster_labels[indices]
259         y_subset = y_true[indices]
260     else:
261         X_subset = X_scaled
262         cluster_subset = cluster_labels
263         y_subset = y_true
264
265     tsne = TSNE(n_components=2, random_state=42, perplexity=30)
266     X_tsne = tsne.fit_transform(X_subset)
267
268     # Create subplots
269     fig, axes = plt.subplots(1, 2, figsize=(15, 6))
270
271     # Plot clusters
272     scatter1 = axes[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=
cluster_subset, cmap='viridis', alpha=0.7)
273     axes[0].set_xlabel('t-SNE Component 1')
274     axes[0].set_ylabel('t-SNE Component 2')
275     axes[0].set_title(f'{method_name} Clusters (t-SNE)')
276     plt.colorbar(scatter1, ax=axes[0])
277
278     # Plot true labels
279     scatter2 = axes[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=
y_subset, cmap='RdYlBu', alpha=0.7)
280     axes[1].set_xlabel('t-SNE Component 1')
281     axes[1].set_ylabel('t-SNE Component 2')
282     axes[1].set_title('True Labels (t-SNE)')
283     plt.colorbar(scatter2, ax=axes[1])
284
285     plt.tight_layout()
286     plt.savefig(f'images/{method_name.lower().replace(" ", "_")
}_tsne_visualization.png',
287               dpi=300, bbox_inches='tight')
288     plt.close()
289
290 def analyze_cluster_characteristics(X, cluster_labels,
feature_names, titles, publishers):
291     """Analyze characteristics of each cluster"""
292     print("Analyzing cluster characteristics...")
293
294     # Create DataFrame with clusters
295     df_clusters = X.copy()
296     df_clusters['Cluster'] = cluster_labels
297     df_clusters['Title'] = titles
298     df_clusters['Publisher'] = publishers

```

A. Source code

```
299
300     # Calculate cluster statistics
301     cluster_stats = []
302
303     for cluster_id in sorted(set(cluster_labels)):
304         if cluster_id == -1: # Skip noise points in DBSCAN
305             continue
306
307         cluster_data = df_clusters[df_clusters['Cluster'] ==
cluster_id]
308
309         stats = {
310             'Cluster': cluster_id,
311             'Size': len(cluster_data),
312             'Percentage': len(cluster_data) / len(df_clusters)
* 100
313         }
314
315         # Add mean values for each feature
316         for feature in feature_names:
317             stats[f'{feature}_mean'] = cluster_data[feature].
mean()
318             stats[f'{feature}_std'] = cluster_data[feature].std
()
319
320         cluster_stats.append(stats)
321
322     # Convert to DataFrame
323     cluster_stats_df = pd.DataFrame(cluster_stats)
324     cluster_stats_df.to_csv('images/cluster_characteristics.csv
', index=False)
325
326     # Create heatmap of cluster means
327     feature_means = cluster_stats_df[[col for col in
cluster_stats_df.columns if col.endswith('_mean')]]
328     feature_means.columns = [col.replace('_mean', '') for col
in feature_means.columns]
329     feature_means.index = cluster_stats_df['Cluster']
330
331     plt.figure(figsize=(12, 8))
332     sns.heatmap(feature_means.T, annot=True, cmap='RdYlBu_r',
center=0, fmt='.2f')
333     plt.title('Cluster Characteristics Heatmap (Mean Values)')
334     plt.xlabel('Cluster')
335     plt.ylabel('Features')
336     plt.tight_layout()
337     plt.savefig('images/cluster_characteristics_heatmap.png',
dpi=300, bbox_inches='tight')
```

```

338 plt.close()
339
340 return cluster_stats_df, df_clusters
341
342 def identify_suspicious_patterns(df_clusters, y_true):
343     """Identify potentially suspicious patterns in clusters"""
344     print("Identifying suspicious patterns...")
345
346     suspicious_patterns = []
347
348     for cluster_id in sorted(set(df_clusters['Cluster'])):
349         if cluster_id == -1: # Skip noise points
350             continue
351
352         cluster_data = df_clusters[df_clusters['Cluster'] ==
cluster_id]
353
354         # Use only indices that exist in y_true
355         valid_indices = [i for i in cluster_data.index if i <
len(y_true)]
356         if len(valid_indices) == 0:
357             continue
358
359         # Calculate proportion of problematic journals in
cluster
360         true_labels_cluster = y_true[valid_indices]
361         problematic_ratio = np.mean(true_labels_cluster == 0.0)
362
363         # Identify suspicious characteristics
364         high_self_citation = cluster_data['Self-Cites/Total
Cites (3years)'].mean() > 0.1
365         high_uncited = cluster_data['Uncited Docs./Total Docs.
(3years)'].mean() > 0.5
366         low_sjr = cluster_data['SJR'].mean() < 1.0
367
368         pattern = {
369             'Cluster': cluster_id,
370             'Size': len(cluster_data),
371             'Problematic_Ratio': problematic_ratio,
372             'High_Self_Citation': high_self_citation,
373             'High_Uncited_Docs': high_uncited,
374             'Low_SJR': low_sjr,
375             'Suspicion_Score': sum([high_self_citation,
high_uncited, low_sjr, problematic_ratio > 0.5])
376         }
377
378         suspicious_patterns.append(pattern)
379

```

A. Source code

```
380     # Convert to DataFrame and sort by suspicion score
381     suspicious_df = pd.DataFrame(suspicious_patterns)
382     suspicious_df = suspicious_df.sort_values('Suspicion_Score',
383     , ascending=False)
384
385     print("\nSuspicious Pattern Analysis:")
386     print(suspicious_df)
387
388     suspicious_df.to_csv('images/suspicious_patterns_analysis.
389     csv', index=False)
390
391     return suspicious_df
392
393 def main():
394     """Main function to run clustering analysis"""
395     # Create images directory
396     import os
397     os.makedirs('images', exist_ok=True)
398
399     print("StAMA - Statistical Analysis in Modern Academia")
400     print("Script 4: Clustering Analysis")
401     print("="*60)
402
403     # Load and prepare data
404     X, y_true, titles, publishers, feature_names =
405     load_and_prepare_data()
406
407     # Scale features
408     X_standard, X_robust, scaler_standard, scaler_robust =
409     scale_features(X)
410
411     # Use robust scaling for main analysis
412     X_scaled = X_robust
413
414     # Determine optimal number of clusters
415     optimal_k, optimization_results =
416     determine_optimal_clusters(X_scaled)
417
418     # Perform different clustering methods
419     print("\n" + "="*40)
420     print("CLUSTERING METHODS COMPARISON")
421     print("="*40)
422
423     # K-means
424     kmeans_model, kmeans_labels, kmeans_silhouette =
425     perform_kmeans_clustering(X_scaled, optimal_k, y_true)
426
427     # DBSCAN
```

```

422     dbscan_labels, best_eps = perform_dbscan_clustering(
423         X_scaled, y_true)
424
425     # Hierarchical
426     hierarchical_model, hierarchical_labels =
427     perform_hierarchical_clustering(X_scaled, optimal_k, y_true)
428
429     # Visualizations
430     print("\n" + "="*40)
431     print("CREATING VISUALIZATIONS")
432     print("="*40)
433
434     # PCA visualizations
435     pca_kmeans, X_pca_kmeans = visualize_clusters_pca(X_scaled,
436         kmeans_labels, y_true, "K-means")
437     visualize_clusters_pca(X_scaled, hierarchical_labels,
438         y_true, "Hierarchical")
439
440     # t-SNE visualizations
441     visualize_clusters_tsne(X_scaled, kmeans_labels, y_true, "K-
442     means")
443     visualize_clusters_tsne(X_scaled, hierarchical_labels,
444         y_true, "Hierarchical")
445
446     # Analyze cluster characteristics (using K-means results)
447     cluster_stats_df, df_clusters =
448     analyze_cluster_characteristics(X, kmeans_labels,
449         feature_names, titles, publishers)
450
451     # Identify suspicious patterns
452     suspicious_df = identify_suspicious_patterns(df_clusters,
453         y_true)
454
455     print("\n" + "="*60)
456     print("CLUSTERING ANALYSIS COMPLETE")
457     print("="*60)
458     print("Results saved in 'images' directory:")
459     print("- optimal_clusters_analysis.png")
460     print("- cluster_optimization_results.csv")
461     print("- *_pca_visualization.png")
462     print("- *_tsne_visualization.png")
463     print("- cluster_characteristics.csv")
464     print("- cluster_characteristics_heatmap.png")
465     print("- suspicious_patterns_analysis.csv")
466
467 if __name__ == "__main__":

```

A. Source code

459 `main()`

Listing A.4: Clustering Analysis

```

1
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.model_selection import train_test_split,
   cross_val_score, GridSearchCV, StratifiedKFold
7 from sklearn.preprocessing import StandardScaler, RobustScaler
8 from sklearn.ensemble import RandomForestClassifier,
   GradientBoostingClassifier
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.svm import SVC
11 from sklearn.naive_bayes import GaussianNB
12 from sklearn.metrics import (classification_report,
   confusion_matrix, roc_auc_score,
13                               roc_curve, precision_recall_curve,
   accuracy_score,
14                               precision_score, recall_score,
   f1_score)
15 from sklearn.feature_selection import SelectKBest, f_classif,
   RFE
16 import warnings
17 warnings.filterwarnings('ignore')
18
19 # Set style for better visualizations
20 plt.style.use('seaborn-v0_8')
21 sns.set_palette("husl")
22
23 def load_and_prepare_data():
24     """Load and prepare data for classification analysis"""
25     print("Loading and preparing data for classification
   analysis...")
26     df = pd.read_csv(r'C:\Users\TEJA\Downloads\tj\tj\
   Data_with_Labels.csv')
27
28     # Select features for classification
29     classification_features = [
30         'SJR', 'H index', 'IPP', 'SNIP',
31         'Self-Cites/Total Cites (3years)',
32         'Uncited Docs./Total Docs. (3years)',
33         'Cites / Doc. (2years)',
34         'Ref. / Doc.',
35         'Total Docs. (2023)',
36         'Total Cites (3years)',
37         'Coverage_Duration'
38     ]
39
40     # Create feature matrix and target vector

```

A. Source code

```
41 X = df[classification_features].copy()
42 y = df['Labels'].copy()
43
44 # Handle missing values
45 X = X.fillna(X.median())
46
47 # Remove rows with missing target values
48 mask = ~y.isna()
49 X = X[mask]
50 y = y[mask]
51
52 # Convert labels to binary (1 for legitimate, 0 for
problematic)
53 y = (y == 1.0).astype(int)
54
55 print(f"Dataset shape: {X.shape}")
56 print(f"Class distribution:")
57 print(f"Legitimate journals (1): {sum(y == 1)} ({sum(y ==
1)/len(y)*100:.1f}%)")
58 print(f"Problematic journals (0): {sum(y == 0)} ({sum(y ==
0)/len(y)*100:.1f}%)")
59
60 return X, y, classification_features
61
62 def feature_selection_analysis(X, y, feature_names):
63     """Perform feature selection analysis"""
64     print("\n" + "="*60)
65     print("FEATURE SELECTION ANALYSIS")
66     print("="*60)
67
68     # Univariate feature selection
69     selector = SelectKBest(score_func=f_classif, k='all')
70     X_selected = selector.fit_transform(X, y)
71
72     # Get feature scores
73     feature_scores = pd.DataFrame({
74         'Feature': feature_names,
75         'Score': selector.scores_,
76         'P_value': selector.pvalues_
77     }).sort_values('Score', ascending=False)
78
79     print("Univariate Feature Selection (F-statistic):")
80     print(feature_scores)
81
82     # Save feature scores
83     feature_scores.to_csv('images/feature_selection_scores.csv'
, index=False)
84
```

```

85     # Plot feature importance
86     plt.figure(figsize=(12, 8))
87     bars = plt.barh(range(len(feature_scores)), feature_scores[
88         'Score'])
89     plt.yticks(range(len(feature_scores)), feature_scores['
90         Feature'])
91     plt.xlabel('F-statistic Score')
92     plt.title('Feature Importance (Univariate F-test)')
93     plt.grid(True, alpha=0.3)
94
95     # Color bars based on p-value significance
96     for i, (bar, p_val) in enumerate(zip(bars, feature_scores['
97         P_value'])):
98         if p_val < 0.001:
99             bar.set_color('red')
100         elif p_val < 0.01:
101             bar.set_color('orange')
102         elif p_val < 0.05:
103             bar.set_color('yellow')
104         else:
105             bar.set_color('gray')
106
107     plt.tight_layout()
108     plt.savefig('images/feature_importance_univariate.png', dpi
109 =300, bbox_inches='tight')
110     plt.close()
111
112     return feature_scores
113
114 def train_classification_models(X_train, X_test, y_train,
115 y_test):
116     """Train and evaluate multiple classification models"""
117     print("\n" + "="*60)
118     print("TRAINING CLASSIFICATION MODELS")
119     print("="*60)
120
121     # Define models
122     models = {
123         'Logistic Regression': LogisticRegression(random_state
124 =42, max_iter=1000),
125         'Random Forest': RandomForestClassifier(random_state
126 =42, n_estimators=100),
127         'Gradient Boosting': GradientBoostingClassifier(
128 random_state=42, n_estimators=100),
129         'SVM': SVC(random_state=42, probability=True),
130         'Naive Bayes': GaussianNB()
131     }

```

A. Source code

```
125     # Store results
126     results = []
127     trained_models = {}
128
129     for name, model in models.items():
130         print(f"\nTraining {name}...")
131
132         # Train model
133         model.fit(X_train, y_train)
134         trained_models[name] = model
135
136         # Make predictions
137         y_pred = model.predict(X_test)
138         y_pred_proba = model.predict_proba(X_test)[: , 1] if
hasattr(model, 'predict_proba') else None
139
140         # Calculate metrics
141         accuracy = accuracy_score(y_test, y_pred)
142         precision = precision_score(y_test, y_pred)
143         recall = recall_score(y_test, y_pred)
144         f1 = f1_score(y_test, y_pred)
145
146         if y_pred_proba is not None:
147             auc_score = roc_auc_score(y_test, y_pred_proba)
148         else:
149             auc_score = np.nan
150
151         # Cross-validation
152         cv_scores = cross_val_score(model, X_train, y_train, cv
=5, scoring='accuracy')
153         cv_mean = cv_scores.mean()
154         cv_std = cv_scores.std()
155
156         print(f"Accuracy: {accuracy:.4f}")
157         print(f"Precision: {precision:.4f}")
158         print(f"Recall: {recall:.4f}")
159         print(f"F1-score: {f1:.4f}")
160         print(f"AUC: {auc_score:.4f}")
161         print(f"CV Accuracy: {cv_mean:.4f} (+/- {cv_std*2:.4f})
")
162
163     # Store results
164     results.append({
165         'Model': name,
166         'Accuracy': accuracy,
167         'Precision': precision,
168         'Recall': recall,
169         'F1_Score': f1,
```

```

170         'AUC': auc_score,
171         'CV_Mean': cv_mean,
172         'CV_Std': cv_std
173     })
174
175     # Convert to DataFrame
176     results_df = pd.DataFrame(results)
177     results_df.to_csv('images/classification_results.csv',
178                     index=False)
179
180     return results_df, trained_models
181
182 def hyperparameter_tuning(X_train, y_train):
183     """Perform hyperparameter tuning for best models"""
184     print("\n" + "="*60)
185     print("HYPERPARAMETER TUNING")
186     print("="*60)
187
188     # Random Forest hyperparameter tuning
189     print("Tuning Random Forest...")
190     rf_param_grid = {
191         'n_estimators': [50, 100, 200],
192         'max_depth': [None, 10, 20, 30],
193         'min_samples_split': [2, 5, 10],
194         'min_samples_leaf': [1, 2, 4]
195     }
196
197     rf_grid = GridSearchCV(
198         RandomForestClassifier(random_state=42),
199         rf_param_grid,
200         cv=5,
201         scoring='f1',
202         n_jobs=-1
203     )
204     rf_grid.fit(X_train, y_train)
205
206     print(f"Best RF parameters: {rf_grid.best_params_}")
207     print(f"Best RF score: {rf_grid.best_score_:.4f}")
208
209     # Gradient Boosting hyperparameter tuning
210     print("\nTuning Gradient Boosting...")
211     gb_param_grid = {
212         'n_estimators': [50, 100, 200],
213         'learning_rate': [0.01, 0.1, 0.2],
214         'max_depth': [3, 5, 7],
215         'min_samples_split': [2, 5, 10]
216     }

```

A. Source code

```
217 gb_grid = GridSearchCV(
218     GradientBoostingClassifier(random_state=42),
219     gb_param_grid,
220     cv=5,
221     scoring='f1',
222     n_jobs=-1
223 )
224 gb_grid.fit(X_train, y_train)
225
226 print(f"Best GB parameters: {gb_grid.best_params_}")
227 print(f"Best GB score: {gb_grid.best_score_:.4f}")
228
229 # Save tuning results
230 tuning_results = {
231     'Random_Forest': {
232         'best_params': rf_grid.best_params_,
233         'best_score': rf_grid.best_score_
234     },
235     'Gradient_Boosting': {
236         'best_params': gb_grid.best_params_,
237         'best_score': gb_grid.best_score_
238     }
239 }
240
241 # Save as DataFrame
242 tuning_df = pd.DataFrame([
243     {'Model': 'Random Forest', 'Best_Score': rf_grid.
244     best_score_, 'Best_Params': str(rf_grid.best_params_)},
245     {'Model': 'Gradient Boosting', 'Best_Score': gb_grid.
246     best_score_, 'Best_Params': str(gb_grid.best_params_)}
247 ])
248 tuning_df.to_csv('images/hyperparameter_tuning_results.csv',
249                 index=False)
250
251 return rf_grid.best_estimator_, gb_grid.best_estimator_
252
253 def create_model_comparison_plots(results_df):
254     """Create comparison plots for different models"""
255     print("Creating model comparison visualizations...")
256
257     # Model performance comparison
258     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
259
260     # Accuracy comparison
261     axes[0, 0].bar(results_df['Model'], results_df['Accuracy'])
262     axes[0, 0].set_title('Model Accuracy Comparison')
263     axes[0, 0].set_ylabel('Accuracy')
264     axes[0, 0].tick_params(axis='x', rotation=45)
```

```

262 axes[0, 0].grid(True, alpha=0.3)
263
264 # F1-score comparison
265 axes[0, 1].bar(results_df['Model'], results_df['F1_Score'])
266 axes[0, 1].set_title('Model F1-Score Comparison')
267 axes[0, 1].set_ylabel('F1-Score')
268 axes[0, 1].tick_params(axis='x', rotation=45)
269 axes[0, 1].grid(True, alpha=0.3)
270
271 # AUC comparison
272 axes[1, 0].bar(results_df['Model'], results_df['AUC'])
273 axes[1, 0].set_title('Model AUC Comparison')
274 axes[1, 0].set_ylabel('AUC')
275 axes[1, 0].tick_params(axis='x', rotation=45)
276 axes[1, 0].grid(True, alpha=0.3)
277
278 # Cross-validation scores
279 axes[1, 1].bar(results_df['Model'], results_df['CV_Mean'])
280 axes[1, 1].errorbar(range(len(results_df)), results_df['
CV_Mean'],
281                    yerr=results_df['CV_Std'], fmt='none',
color='red', capsize=5)
282 axes[1, 1].set_title('Cross-Validation Accuracy')
283 axes[1, 1].set_ylabel('CV Accuracy')
284 axes[1, 1].tick_params(axis='x', rotation=45)
285 axes[1, 1].grid(True, alpha=0.3)
286
287 plt.tight_layout()
288 plt.savefig('images/model_comparison_plots.png', dpi=300,
bbox_inches='tight')
289 plt.close()
290
291 def create_roc_curves(trained_models, X_test, y_test):
292     """Create ROC curves for all models"""
293     print("Creating ROC curves...")
294
295     plt.figure(figsize=(10, 8))
296
297     for name, model in trained_models.items():
298         if hasattr(model, 'predict_proba'):
299             y_pred_proba = model.predict_proba(X_test)[: , 1]
300             fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
301             auc_score = roc_auc_score(y_test, y_pred_proba)
302             plt.plot(fpr, tpr, label=f'{name} (AUC = {auc_score
:.3f})')
303
304     plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
305     plt.xlabel('False Positive Rate')

```

A. Source code

```
306     plt.ylabel('True Positive Rate')
307     plt.title('ROC Curves Comparison')
308     plt.legend()
309     plt.grid(True, alpha=0.3)
310     plt.savefig('images/roc_curves_comparison.png', dpi=300,
311               bbox_inches='tight')
312     plt.close()
313
314 def create_confusion_matrices(trained_models, X_test, y_test):
315     """Create confusion matrices for all models"""
316     print("Creating confusion matrices...")
317
318     n_models = len(trained_models)
319     fig, axes = plt.subplots(2, 3, figsize=(18, 12))
320     axes = axes.ravel()
321
322     for i, (name, model) in enumerate(trained_models.items()):
323         y_pred = model.predict(X_test)
324         cm = confusion_matrix(y_test, y_pred)
325
326         sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=
327 axes[i])
328         axes[i].set_title(f'{name} Confusion Matrix')
329         axes[i].set_xlabel('Predicted')
330         axes[i].set_ylabel('Actual')
331
332     # Remove empty subplot
333     if n_models < len(axes):
334         fig.delaxes(axes[-1])
335
336     plt.tight_layout()
337     plt.savefig('images/confusion_matrices.png', dpi=300,
338               bbox_inches='tight')
339     plt.close()
340
341 def feature_importance_analysis(best_model, feature_names):
342     """Analyze feature importance from the best model"""
343     print("Analyzing feature importance...")
344
345     if hasattr(best_model, 'feature_importances_'):
346         importances = best_model.feature_importances_
347
348         # Create feature importance DataFrame
349         feature_importance_df = pd.DataFrame({
350             'Feature': feature_names,
351             'Importance': importances
352         }).sort_values('Importance', ascending=False)
```

```

351     print("Feature Importance (from best model):")
352     print(feature_importance_df)
353
354     # Save feature importance
355     feature_importance_df.to_csv('images/
feature_importance_best_model.csv', index=False)
356
357     # Plot feature importance
358     plt.figure(figsize=(12, 8))
359     bars = plt.barh(range(len(feature_importance_df)),
feature_importance_df['Importance'])
360     plt.yticks(range(len(feature_importance_df)),
feature_importance_df['Feature'])
361     plt.xlabel('Feature Importance')
362     plt.title('Feature Importance from Best Model')
363     plt.grid(True, alpha=0.3)
364
365     # Color bars based on importance
366     max_importance = feature_importance_df['Importance'].
max()
367     for bar, importance in zip(bars, feature_importance_df[
'Importance']):
368         if importance > 0.7 * max_importance:
369             bar.set_color('red')
370         elif importance > 0.4 * max_importance:
371             bar.set_color('orange')
372         elif importance > 0.2 * max_importance:
373             bar.set_color('yellow')
374         else:
375             bar.set_color('lightblue')
376
377     plt.tight_layout()
378     plt.savefig('images/feature_importance_best_model.png',
dpi=300, bbox_inches='tight')
379     plt.close()
380
381     return feature_importance_df
382 else:
383     print("Best model does not have feature_importances_
attribute")
384     return None
385
386 def predict_suspicious_journals(best_model, X, y, feature_names
):
387     """Use the best model to identify potentially suspicious
journals"""
388     print("Identifying potentially suspicious journals...")
389

```

A. Source code

```
390     # Get predictions and probabilities
391     y_pred = best_model.predict(X)
392     y_pred_proba = best_model.predict_proba(X)[: , 0]  #
Probability of being problematic
393
394     # Create results DataFrame
395     df = pd.read_csv('Data_with_Labels.csv')
396
397     # Filter to match our processed data
398     df_filtered = df.dropna(subset=['Labels'])
399
400     # Only fill numeric columns with median
401     numeric_columns = df_filtered.select_dtypes(include=[np.
number]).columns
402     df_filtered[numeric_columns] = df_filtered[numeric_columns
].fillna(df_filtered[numeric_columns].median())
403
404     results_df = df_filtered[['Title', 'Publisher', 'Labels']].
copy()
405     results_df['Predicted_Label'] = y_pred
406     results_df['Problematic_Probability'] = y_pred_proba
407     results_df['True_Positive'] = (results_df['Labels'] == 0.0)
& (results_df['Predicted_Label'] == 0)
408     results_df['False_Positive'] = (results_df['Labels'] ==
1.0) & (results_df['Predicted_Label'] == 0)
409
410     # Sort by problematic probability
411     results_df = results_df.sort_values('
Problematic_Probability', ascending=False)
412
413     # Save results
414     results_df.to_csv('images/journal_predictions.csv', index=
False)
415
416     # Print top suspicious journals
417     print("\nTop 10 journals flagged as potentially problematic
:")
418     print(results_df.head(10)[['Title', 'Publisher', '
Problematic_Probability', 'Labels']])
419
420     return results_df
421
422 def main():
423     """Main function to run classification analysis"""
424     # Create images directory
425     import os
426     os.makedirs('images', exist_ok=True)
427
```

```

428 print("StAMA - Statistical Analysis in Modern Academia")
429 print("Script 5: Classification Analysis")
430 print("="*60)
431
432 # Load and prepare data
433 X, y, feature_names = load_and_prepare_data()
434
435 # Split data
436 X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
437
438 # Scale features
439 scaler = RobustScaler()
440 X_train_scaled = scaler.fit_transform(X_train)
441 X_test_scaled = scaler.transform(X_test)
442
443 # Feature selection analysis
444 feature_scores = feature_selection_analysis(X_train_scaled,
y_train, feature_names)
445
446 # Train models
447 results_df, trained_models = train_classification_models(
X_train_scaled, X_test_scaled, y_train, y_test)
448
449 # Hyperparameter tuning
450 best_rf, best_gb = hyperparameter_tuning(X_train_scaled,
y_train)
451
452 # Evaluate best models
453 best_model = best_rf if results_df[results_df['Model'] == '
Random Forest']['F1_Score'].iloc[0] > \
454 results_df[results_df['Model'] == '
Gradient Boosting']['F1_Score'].iloc[0] else best_gb
455
456 # Create visualizations
457 create_model_comparison_plots(results_df)
458 create_roc_curves(trained_models, X_test_scaled, y_test)
459 create_confusion_matrices(trained_models, X_test_scaled,
y_test)
460
461 # Feature importance analysis
462 feature_importance_df = feature_importance_analysis(
best_model, feature_names)
463
464 # Predict suspicious journals
465 X_scaled = scaler.fit_transform(X)
466 predictions_df = predict_suspicious_journals(best_model,
X_scaled, y, feature_names)

```

A. Source code

```
467     print("\n" + "="*60)
468     print("CLASSIFICATION ANALYSIS COMPLETE")
469     print("="*60)
470     print("Results saved in 'images' directory:")
471     print("- feature_selection_scores.csv")
472     print("- feature_importance_univariate.png")
473     print("- classification_results.csv")
474     print("- hyperparameter_tuning_results.csv")
475     print("- model_comparison_plots.png")
476     print("- roc_curves_comparison.png")
477     print("- confusion_matrices.png")
478     print("- feature_importance_best_model.csv and .png")
479     print("- journal_predictions.csv")
480
481
482 if __name__ == "__main__":
483     main()
```

Listing A.5: Classification Analysis

Bibliography

- M. Baldwin. Origins of the journal impact factor. *Institute for Scientific Information*, 1964.
- J. Bohannon. Who's afraid of peer review? *American Association for the Advancement of Science*, 342, 2013.
- C. L. Guillaume Cabanac and A. Magazinov. Tortured phrases: A dubious writing style emerging in science. evidence of critical issues affecting established journals. *HAL Open Science*, 2021.
- J. Hirsch. An index to quantify an individual's scientific research output. *Physics and Society*, 2005.
- K. G. C. R. M. D. M. R. A. H. A. H. Jingfeng Xia, Jennifer L. Harmon. Who publishes in “predatory” journals? *Journal of the association for information science and technology*, 66:1406:1417, 2015.
- C. Shen and B.-C. Björk. 'Predatory' open access: A longitudinal study of article volumes and market characteristics. *BMC Medicine*, 48(1-2):214:230, 2015.